GUIA DE ESTUDO

# Desafio Backend Go Processamento Distribuído de Apostas

O que foi construído em cada spec, por que foi feito assim e quais conceitos de Go aparecem, para quem vem de C#, Python e TypeScript.

---

Repositório: github.com/rafaelcorreadev1/junglegaming
Atualizado até a spec 12 · 13/10/2026

# Como usar este guia

Cada spec concluída e testada vira um capítulo. Todos os capítulos seguem a mesma ordem:

1. **O que a spec pedia** — o requisito do desafio.
2. **O que foi construído** — os arquivos e o papel de cada um.
3. **Como funciona** — o fluxo, passo a passo.
4. **Conceitos de Go** — o que é novo, comparado com C# e TypeScript.
5. **Decisões** — as escolhas feitas e o motivo.
6. **Como foi verificado** — o que foi executado e o resultado.
7. **Perguntas de revisão** — para testar se você entendeu.

Leia com o código aberto ao lado. Os trechos aqui são recortes; o arquivo completo está no caminho indicado.

## Progresso

| # | SPEC | STATUS |
|---|---|---|
| 01 | Bootstrap: módulo Go, Fx, config, Docker Compose | Concluída |
| 02 | Money | Concluída |
| 03 | Domínio: Wallet, Ledger, WagerTransaction | Concluída |
| 04 | Schema e migrations | Concluída |
| 05 | Persistência e transação SQL | Concluída |
| 06 | Carteira: abertura, leitura, ledger paginado | Concluída |
| 07 | Operação: BET / WIN / LOSS e idempotência | Concluída |
| 08 | Reversões e referências pendentes | Concluída |
| 09 | Autenticação e autorização | Concluída |
| 10 | Outbox e eventos | Concluída |
| 11 | Consumidor SQS, inbox e DLQ | Concluída |
| 12 | Reconciliação | Concluída |
| 13 | Observabilidade | A fazer |
| 14 | Ciclo de vida Fx e shutdown | A fazer |
| 15 | Testes de integração, concorrência e recuperação | A fazer |
| 16 | Documentação e entrega | A fazer |

## O desafio em um parágrafo

Um serviço que movimenta carteiras de jogadores a partir de operações enviadas por provedores de jogos: aposta ( `BET` ), ganho ( `WIN` ), perda ( `LOSS` ), devolução ( `REFUND` ) e estorno ( `ROLLBACK` ). As operações chegam por HTTP e por fila (SQS), e o resultado financeiro precisa ser o mesmo nos dois caminhos, mesmo com várias instâncias rodando e com falhas no meio do processamento.

## Mapa mental: Go para quem vem de C# e TypeScript

| EM GO | O EQUIVALENTE QUE VOCÊ CONHECE |
|---|---|
| `go.mod` / `go.sum` | `.csproj` / `package.json` + lockfile |
| Pasta `internal/` | `internal` do C#: o compilador impede import de fora do módulo |
| Nome com maiúscula | `public` ; minúscula é privado ao pacote |
| `struct` + função `NewX` | Classe + construtor |
| `interface` implícita | Interface estrutural do TypeScript: não existe `implements` |
| `error` como retorno | Exceção, só que retornada e checada com `if err != nil` |
| `context.Context` | `CancellationToken` |
| `defer` | `finally` / `using` |
| Goroutine ( `go f()` ) | `Task.Run` |
| Uber Fx | `IServiceCollection` / modules do NestJS |

SPEC 01

# Bootstrap: módulo Go, Fx, config e Docker Compose

Um processo Go que sobe, valida a configuração, conecta no Postgres e responde dois endpoints de saúde. Ainda não há nenhuma regra de negócio.

## 1. O que a spec pedia

- Versão do Go declarada em `go.mod` e no `Dockerfile`; dependências versionadas.
- Composição da aplicação com Uber Fx, com injeção por construtores.
- Configuração validada na inicialização: o processo não sobe com config inválida.
- Ambiente local com Docker Compose e um `.env.example` sem segredos.

## 2. O que foi construído

| ARQUIVO | PAPEL |
|---|---|
| `cmd/app/main.go` | Ponto de entrada: carrega a config e entrega o controle ao Fx |
| `internal/config/config.go` | Lê e valida as variáveis de ambiente |
| `internal/app/app.go` | A composição: lista os módulos que formam a aplicação |
| `internal/infra/observability/logger.go` | Logger em JSON |
| `internal/infra/postgres/pool.go` | Pool de conexões com o Postgres |
| `internal/infra/httpapi/server.go` | Rotas e servidor HTTP |
| `internal/infra/httpapi/health.go` | Endpoints `/health/live` e `/health/ready` |
| `Dockerfile`, `docker-compose.yml` | Imagem da aplicação e ambiente local |

## 3. Como funciona

1. `main` lê as variáveis de ambiente e monta a `Config`.
2. Config inválida: imprime o erro e sai com código 1. Nada mais é criado.
3. O Fx recebe os construtores e descobre a ordem: logger, pool, handler, router, servidor.
4. O Fx executa os `OnStart`: ping no banco e depois abertura da porta HTTP.
5. O processo fica parado esperando um sinal (`SIGTERM` ou Ctrl+C).
6. No sinal, o Fx executa os `OnStop` na ordem inversa: o servidor HTTP para e depois o pool fecha.

## 4. Conceitos de Go

### Erro é valor de retorno

Go não tem exceção. A função que pode falhar devolve o erro como último retorno e quem chama é obrigado a decidir o que fazer com ele.

```
cfg, err := config.Load()
if err != nil {
    fmt.Fprintln(os.Stderr, err)
    os.Exit(1)
}
```

### Embrulhar erro com `%w`

O erro original fica guardado dentro do novo, como a `InnerException` do C#. É o que permite, mais à frente, classificar erros com `errors.Is`, uma exigência do desafio.

```
if err := pool.Ping(ctx); err != nil {
    return fmt.Errorf("ping postgres: %w", err)
}
```

### Visibilidade pela primeira letra

`Config`, `Load` e `NewPool` começam com maiúscula e são públicos. `valueOr` e `writeJSON` começam com minúscula e só existem dentro do próprio pacote.

### Struct, construtor e método

Não há classes. Os dados ficam em uma `struct`, o construtor é uma função comum chamada `NewAlgo` por convenção, e o método é uma função com um *receiver*, que faz o papel do `this`.

```
type HealthHandler struct {
    pool   *pgxpool.Pool
    logger *slog.Logger
}

func NewHealthHandler(pool *pgxpool.Pool, logger *slog.Logger) *HealthHandler {
    return &HealthHandler{pool: pool, logger: logger}
}

func (h *HealthHandler) Live(w http.ResponseWriter, _ *http.Request) {
    writeJSON(w, http.StatusOK, map[string]string{"status": "ok"})
}
```

O `*` indica ponteiro (referência ao mesmo objeto) e o `&` cria esse ponteiro. O `_` significa "recebo o parâmetro, mas não uso".

### `context.Context` e `defer`

O `context` vai sempre como primeiro parâmetro e carrega cancelamento e prazo. No readiness, ele limita o ping a 2 segundos. O `defer` agenda a linha para o fim da função, com ou sem erro.

```
ctx, cancel := context.WithTimeout(r.Context(), readinessTimeout)
defer cancel()

if err := h.pool.Ping(ctx); err != nil { ... }
```

#### Goroutine

`go func() { ... }()` inicia uma execução concorrente. O servidor HTTP roda em uma goroutine porque `Serve` bloqueia até o servidor fechar, e a inicialização precisa continuar.

## 5. Uber Fx

O Fx é o container de injeção de dependência. Você não registra "interface para implementação": entrega as funções construtoras e ele lê as assinaturas.

```
func NewRouter(health *HealthHandler) http.Handler
```

O Fx lê isso como: para criar um `http.Handler`, preciso de um `*HealthHandler`. Ele procura quem produz esse tipo e chama na ordem certa.

| FUNÇÃO | O QUE FAZ |
|---|---|
| `fx.Provide(NewX)` | Registra um construtor. Só executa se alguém precisar do resultado. |
| `fx.Invoke(f)` | Executa sempre. É o que força a construção do servidor. |
| `fx.Supply(valor)` | Registra um valor pronto, sem construtor. Usado para a `Config`. |
| `fx.Module("nome", ...)` | Agrupa os registros de um pacote. |
| `fx.Lifecycle` | Onde cada componente registra `OnStart` e `OnStop`. |

A composição inteira, em `internal/app/app.go`:

```
func Options(cfg config.Config) fx.Option {
    return fx.Options(
        fx.Supply(cfg),
        fx.StartTimeout(cfg.ShutdownTimeout),
        fx.StopTimeout(cfg.ShutdownTimeout),
        fx.WithLogger(...),
        observability.Module,
        postgres.Module,
        httpapi.Module,
    )
}
```

E o ciclo de vida do pool, em `internal/infra/postgres/pool.go`:

```
lc.Append(fx.Hook{
    OnStart: func(ctx context.Context) error {
        if err := pool.Ping(ctx); err != nil {
            return fmt.Errorf("ping postgres: %w", err)
        }
        return nil
    },
    OnStop: func(context.Context) error {
        pool.Close()
        return nil
    },
})
```

**Por que a ordem de parada sai certa sozinha.** O servidor depende do handler, que depende do pool. Então o pool é construído primeiro e registra seu hook primeiro. O Fx executa os `OnStop` na ordem inversa dos `OnStart`: o servidor para de receber requisições antes de o pool fechar.

## 6. Decisões

| DECISÃO | MOTIVO |
|---|---|
| A config devolve todos os erros de uma vez (`errors.Join`) | Quem configura o ambiente corrige tudo em uma tentativa |
| A config recebe uma função `getenv` em vez de chamar `os.Getenv` | O teste passa um mapa e não altera o ambiente real |
| A config é carregada no `main`, antes do Fx | Os timeouts do próprio Fx vêm dela |
| A porta é aberta no `OnStart`, fora da goroutine | Porta ocupada vira falha de inicialização, em vez de erro perdido em background |
| O ping no banco fica no `OnStart` | Criar o pool não abre conexão; só o ping valida o banco |
| Composição em `internal/app`, fora do `main` | Permite testar o grafo de dependências sem subir nada |
| Dockerfile em dois estágios, com imagem final *distroless* | Go gera um binário único; a imagem final não precisa de compilador nem de shell |

### Liveness e readiness

| ENDPOINT | PERGUNTA QUE RESPONDE | VERIFICA |
|---|---|---|
| `GET /health/live` | O processo está de pé? | Nada externo |
| `GET /health/ready` | Consigo atender requisições? | Postgres (o SQS entra na spec 10) |

Se o banco cair, o orquestrador para de mandar tráfego (ready falha), mas não reinicia o processo (live continua ok), porque reiniciar não resolveria.

## 7. Testes

O arquivo de teste fica ao lado do código, com sufixo `_test.go`, e usa só o pacote `testing` da biblioteca padrão. O formato idiomático é o teste em tabela, equivalente ao `[Theory]` do xUnit:

```
tests := []struct {
    name    string
    env     map[string]string
    wantErr string
}{
    {name: "missing database url", env: map[string]string{}, wantErr: "DATABASE_URL is
required"},
    ...
}

for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        _, err := FromEnv(envFrom(tt.env))
        ...
    })
}
```

`internal/app/app_test.go` usa `fx.ValidateApp`: confere que todo construtor tem as dependências de que precisa, sem iniciar nada.

## 8. Como foi verificado

| VERIFICAÇÃO | RESULTADO |
|---|---|
| `gofmt -l .` e `go vet ./...` | Sem saída |
| `go test ./...` | Passou |
| `go test -race ./...` (em container Linux) | Passou |
| `docker compose up --build` | `/health/live` e `/health/ready` respondem 200 |
| Postgres parado | `/health/ready` 503, `/health/live` 200 |
| Sem `DATABASE_URL` e com `LOG_LEVEL` inválido | Os dois erros impressos, código de saída 1 |
| `docker compose stop app` | Servidor para, pool fecha, código de saída 0 |

> **O `-race` não roda direto no Windows.** O detector de condição de corrida exige um compilador C, que não vem com o Go no Windows. A saída é executar em container:
>
> `docker run --rm -v "${PWD}:/src" -v wagering-gomod:/go/pkg/mod -w /src golang:1.27 go test -race ./...`

## 9. Comandos do dia a dia

| COMANDO | PARA QUÊ |
|---|---|
| `go run ./cmd/app` | Roda a aplicação (precisa de `DATABASE_URL`) |
| `go test ./...` | Roda todos os testes; `./...` é "este diretório e todos abaixo" |
| `go vet ./...` | Análise estática |
| `gofmt -l .` | Lista arquivos fora do padrão de formatação |
| `go mod tidy` | Ajusta `go.mod` e `go.sum` ao que o código importa |
| `docker compose up --build` | Sobe Postgres e aplicação |

## 10. Perguntas de revisão

**Por que existe um `fx.Invoke(func(*http.Server) {})` com corpo vazio?**

Porque o Fx só constrói o que alguém pede. Sem essa linha ninguém pediria o servidor, e ele nunca seria criado nem iniciado.

**O que acontece se a porta 8080 já estiver em uso?**

O `net.Listen` falha dentro do `OnStart`, o Fx aborta a inicialização e executa os `OnStop` do que já tinha iniciado (o pool fecha).

**Por que o pool é fechado depois do servidor HTTP no shutdown?**

Porque o servidor ainda pode estar terminando requisições que usam o banco. Fechar o pool antes quebraria essas requisições.

**Qual a diferença entre `fx.Provide` e `fx.Supply`?**

`Provide` registra uma função que cria o valor; `Supply` registra o valor já pronto.

**Por que `FromEnv` recebe uma função em vez de ler o ambiente diretamente?**

Para o teste controlar as variáveis sem alterar o ambiente do processo, o que também permite rodar testes em paralelo.

**Com o banco fora do ar, por que `/health/live` continua 200?**

Liveness diz se o processo precisa ser reiniciado. Banco fora não se resolve reiniciando a aplicação; quem sinaliza isso é o readiness.

### Ordem de leitura do código

1. `cmd/app/main.go`
2. `internal/config/config.go` e o teste ao lado
3. `internal/app/app.go`
4. `internal/infra/postgres/pool.go`
5. `internal/infra/httpapi/server.go` e `health.go`

SPEC 02

# Money

O tipo que representa dinheiro em todo o sistema: centavos em `int64` mais a moeda. Nenhum valor monetário passa por ponto flutuante.

## 1. O que a spec pedia

- Value object imutável com valor e moeda.
- Criação a partir de string decimal, zero por moeda, soma, subtração, negação, comparação e serialização.
- Proibido `float32` e `float64` em parsing, cálculo, serialização e persistência. É critério eliminatório.
- Escala fixa de duas casas e moeda ISO 4217.
- Rejeitar vazio, `NaN`, `Infinity`, notação científica e casas a mais, sem arredondar.
- Operações só entre valores da mesma moeda.
- Tratar overflow no parsing, na soma, na subtração e na negação.

## 2. Por que não usar float

Ponto flutuante guarda números em base 2, e a maioria das frações decimais não tem representação exata nela. Em qualquer linguagem, `0.1 + 0.2` resulta em `0.30000000000000004`. Em um sistema financeiro, esse resíduo vira centavo sobrando ou faltando depois de milhares de operações.

A saída é guardar a quantia como inteiro na menor unidade da moeda: `25.00` vira `2500` centavos. Inteiros somam e subtraem de forma exata.

## 3. O que foi construído

| ARQUIVO | PAPEL |
|---|---|
| `internal/domain/money.go` | O tipo `Money`, seus erros e o parser |
| `internal/domain/money_test.go` | Testes em tabela; 100% das instruções cobertas |

O pacote `domain` importa só a biblioteca padrão. Essa é a regra do desafio: o domínio não conhece Fx, HTTP, SQS nem banco.

### A interface pública

| FUNÇÃO OU MÉTODO | O QUE FAZ |
|---|---|
| `ParseMoney("25.00", "BRL")` | Cria a partir do contrato externo (string decimal) |
| `MoneyFromMinorUnits(2500, "BRL")` | Cria a partir de centavos; é o caminho de volta do banco |
| `ZeroMoney("BRL")` | Zero na moeda informada |
| `Add` , `Sub` , `Neg` | Devolvem um novo `Money` e um erro |
| `Cmp` , `Equal` | Comparação; `Cmp` devolve -1, 0 ou 1 |
| `IsZero` , `IsPositive` , `IsNegative` | Perguntas que as regras de negócio vão usar |
| `Amount()` , `MinorUnits()` , `Currency()` | Leitura: `"25.00"` , `2500` , `"BRL"` |
| `MarshalJSON` , `UnmarshalJSON` | Conversão de e para `{"amount":"25.00","currency":"BRL"}` |

## 4. Como funciona

### Imutabilidade

```
type Money struct {
    minorUnits int64
    currency   string
}
```

Os dois campos começam com minúscula, então nenhum outro pacote consegue alterá-los. Os métodos usam receiver por valor ( `func (m Money)` , sem `*` ): cada chamada trabalha em uma cópia, e o resultado é sempre um `Money` novo. É o equivalente a um `readonly record struct` do C#.

### O parser, passo a passo

1. Separa o sinal `-` , se houver. O sinal `+` não é aceito.
2. Divide no ponto: parte inteira e parte decimal.
3. Exige só dígitos de `0` a `9` nas duas partes. Isso elimina `NaN` , `Infinity` , `1e3` , espaços e vírgulas de uma vez.
4. Rejeita mais de duas casas decimais. Não há arredondamento.
5. Completa a parte decimal com zeros: `"25.5"` vira `"2550"` .
6. Converte dígito a dígito para `int64` , checando overflow antes de cada multiplicação.

Não se usa `strconv.ParseFloat` nem nenhuma função de conversão pronta: o parser é uma validação de caracteres seguida de aritmética de inteiros.

### Overflow

Em Go, somar além do limite de um `int64` não gera erro: o número "dá a volta" e vira negativo, em silêncio. Por isso a checagem acontece **antes** da operação:

```
a, b := m.minorUnits, other.minorUnits
if (b > 0 && a > math.MaxInt64-b) || (b < 0 && a < math.MinInt64-b) {
    return Money{}, fmt.Errorf("%w: %s + %s", ErrMoneyOverflow, m, other)
}
return Money{minorUnits: a + b, currency: m.currency}, nil
```

A leitura é: "se `b` é positivo e `a` já está a menos de `b` do teto, a soma estouraria". A comparação é reescrita de forma que ela própria nunca estoure.

> **O caso da negação.** Um `int64` vai de -9.223.372.036.854.775.808 a +9.223.372.036.854.775.807: existe um negativo a mais. Negar o menor valor não tem resultado possível, então `Neg()` devolve erro nesse caso.

### Erros classificáveis

```
var (
    ErrInvalidMoney     = errors.New("money is not initialized")
    ErrInvalidAmount    = errors.New("invalid money amount")
    ErrInvalidCurrency  = errors.New("invalid currency")
    ErrCurrencyMismatch = errors.New("currency mismatch")
    ErrMoneyOverflow    = errors.New("money amount overflow")
)
```

São os *erros sentinela*: variáveis públicas que servem de identidade do erro. Quem recebe o erro pergunta qual é com `errors.Is`, mesmo que ele tenha sido embrulhado com detalhes pelo `%w`:

```
_, err := brl.Add(usd)
if errors.Is(err, domain.ErrCurrencyMismatch) { ... }
```

Em C# isso seria um `catch (CurrencyMismatchException)`. Mais à frente, a camada HTTP usa essa classificação para decidir entre 400, 409 e 422.

### O valor zero de uma struct

Em Go, toda variável nasce com um "valor zero": `var m Money` é válido para o compilador e contém `0` centavos e moeda vazia. Não existe `null` para structs. Como o desafio exige rejeitar valores não inicializados, a moeda vazia é o marcador: `IsValid()` devolve falso, e toda operação e a serialização devolvem `ErrInvalidMoney`.

### JSON

O pacote `encoding/json` procura dois métodos no tipo. Se existirem, usa-os no lugar do comportamento padrão. É o equivalente a um `JsonConverter` do C#.

```
func (m Money) MarshalJSON() ([]byte, error)      // Money  -> JSON
func (m *Money) UnmarshalJSON(data []byte) error  // JSON   -> Money
```

`UnmarshalJSON` é o único método com receiver de ponteiro (`*Money`), porque precisa preencher a variável de quem chamou. Se o conteúdo for inválido, a variável não é alterada.

O campo `amount` é declarado como `string`. Se o cliente mandar um número JSON (`"amount": 25.00`, sem aspas), a decodificação falha.

## 5. Decisões

| DECISÃO | MOTIVO |
|---|---|
| `int64` em centavos, sem biblioteca decimal | Sem dependência externa; mapeia direto para `BIGINT` no Postgres |
| Limite: cerca de 92 quatrilhões por valor | É o teto do `int64` dividido por 100; fica documentado como limitação |
| Aceita `"25"`, `"25.0"` e `"25.00"`; todas viram `"25.00"` | São o mesmo valor, sem perda. A forma normalizada é a que entra no hash de idempotência (spec 07) |
| Rejeita `"25.000"`, mesmo sendo o mesmo valor | O desafio pede rejeição de escala excedente; aceitar zeros extras abriria exceção à regra |
| Rejeita zeros à esquerda, `+`, `-0.00`, `.50` e `25.` | Uma única forma de escrever cada valor reduz ambiguidade |
| Só moedas ISO 4217 de duas casas | A escala é fixa em 2; `JPY`, sem centavos, ficaria errado |
| Moeda em minúsculas é rejeitada | Não normalizar em silêncio |
| `ParseMoney` aceita negativo | Diferenças negativas são serializadas (reconciliação). Proibir negativo em aposta e saldo é regra das entidades, na spec 03 |

## 6. Conceitos de Go

| CONCEITO | ONDE APARECE | PARALELO |
|---|---|---|
| Receiver por valor ou por ponteiro | `(m Money)` nos métodos; `(m *Money)` em `UnmarshalJSON` | Passagem por cópia ou por referência (`ref`) |
| Valor zero | `Money{}` é inválido de propósito | `default(T)` do C# |
| Erro sentinela + `errors.Is` | `ErrInvalidAmount` e os demais | `catch` por tipo de exceção |
| Múltiplos retornos | `Add` devolve `(Money, error)` | Tupla, ou padrão `TryParse` |
| Struct tags | `` `json:"amount"` `` | `[JsonPropertyName("amount")]` |
| Conjunto com `map[string]struct{}` | Lista de moedas aceitas | `HashSet<string>`; `struct{}` ocupa zero bytes |
| `t.Helper()` | `mustMoney` nos testes | Faz a falha apontar para a linha do teste, não para o helper |

## 7. Como foi verificado

| VERIFICAÇÃO | RESULTADO |
|---|---|
| `go test ./internal/domain/ -cover` | Passou; 100% das instruções |
| `go test -race ./...` (em container) | Passou |
| `gofmt -l .` e `go vet ./...` | Sem saída |
| Busca por `float32` e `float64` nos arquivos `.go` | Nenhuma ocorrência |

### O que os testes cobrem

- **Aceitos:** 12 formas válidas, incluindo os dois extremos do `int64`.
- **Rejeitados:** 23 entradas inválidas, de `NaN` a dígitos não ASCII.
- **Overflow:** no parsing, na soma, na subtração e na negação, incluindo os casos de fronteira que *não* devem estourar.
- **Moedas:** soma, subtração e comparação entre `BRL` e `USD` devolvem `ErrCurrencyMismatch`.
- **Não inicializado:** `Money{}` é rejeitado em sete operações.
- **JSON:** ida e volta, e oito corpos inválidos.

## 8. Perguntas de revisão

### Por que `Add` devolve erro, se somar dois inteiros nunca falha em Go?

Por dois motivos: as moedas podem ser diferentes, e a soma pode passar do limite do `int64`. Go não avisa quando isso acontece; o valor dá a volta e fica negativo.

### Por que `"25.000"` é rejeitado, se vale o mesmo que `"25.00"`?

A escala é fixa em duas casas e o desafio manda rejeitar escala excedente. Aceitar só quando os dígitos extras são zero seria uma exceção difícil de explicar no contrato.

### Como o código garante que ninguém altera um `Money` depois de criado?

Os campos não são exportados e os métodos têm receiver por valor, então sempre devolvem um `Money` novo.

### O que acontece com `var m Money` seguido de `m.Add(outro)`?

Devolve `ErrInvalidMoney`. O valor zero da struct tem moeda vazia, e isso é tratado como "não inicializado".

### Por que `amount` é string no JSON, e não número?

Um número JSON seria lido como ponto flutuante por quase todo parser, inclusive no cliente. Como string, o valor chega exatamente como foi escrito.

### Por que `ParseMoney` aceita `"-5.00"`, se o desafio proíbe valor negativo nas entradas?

`Money` também representa diferenças, que podem ser negativas. Quem proíbe negativo em uma aposta ou em um saldo é a entidade que recebe o valor, na spec 03.

SPEC 03

# Domínio: Wallet, Ledger e WagerTransaction

As regras financeiras do sistema, em Go puro. Nada aqui conhece banco, HTTP ou fila: são structs, métodos e testes que rodam em milissegundos.

## 1. O que a spec pedia

- Entidades com estado encapsulado, construtores com validação e métodos explícitos de transição.
- Criação e reidratação separadas. Reidratar não pode movimentar saldo nem transicionar estado.
- Erros classificáveis com `errors.Is` e `errors.As`; sem `panic` para regra de negócio.
- Carteira que nunca fica negativa e cuja versão só sobe quando o saldo muda.
- Lançamento de ledger imutável, que valida `saldoDepois = saldoAntes ± valor`.
- Transação com máquina de estados e as regras de `BET`, `WIN`, `LOSS`, `REFUND` e `ROLLBACK`.
- Um `failureCode` estável para cada rejeição.

## 2. O que foi construído

| ARQUIVO | CONTEÚDO |
|---|---|
| `errors.go` | Erros sentinela, `ValidationError` e a lista de `FailureCode` |
| `ledger.go` | `LedgerEntry`: um lançamento de débito ou crédito |
| `wallet.go` | `Wallet`: saldo, versão, `Debit` e `Credit` |
| `wager_transaction.go` | `WagerTransaction`: dados da operação e a máquina de estados |
| `wager_rules.go` | `ApplyWager`: as regras dos cinco tipos |

Todos ficam em `internal/domain/`, cada um com o seu `_test.go`.

### Como as três peças se relacionam

| PEÇA | RESPONDE A | MUDA DEPOIS DE CRIADA? |
|---|---|---|
| `WagerTransaction` | O que o provedor pediu e qual foi o resultado? | Só o estado, até ficar terminal |
| `Wallet` | Quanto o jogador tem agora? | Sim, por `Debit` e `Credit` |
| `LedgerEntry` | Como o saldo chegou até aqui? | Nunca |

## 3. Wallet

```
type Wallet struct {
    id        string
    playerID  string
    balance   Money
    version   int64
    createdAt time.Time
    updatedAt time.Time
}
```

Todos os campos são privados. O saldo só muda por `Debit` e `Credit`, e os dois devolvem o lançamento de ledger correspondente. Assim é impossível alterar o saldo sem gerar o registro.

```
func (w *Wallet) Debit(entryID, transactionID string, amount Money, now time.Time) (LedgerEntry,
error)
```

### Validar tudo antes de alterar

O método calcula o novo saldo, checa as regras, monta o lançamento e só então grava. Se qualquer passo falhar, a carteira fica intacta:

```
if newBalance.IsNegative() {
    return LedgerEntry{}, fmt.Errorf("%w: balance %s, debit %s", ErrInsufficientFunds,
w.balance, amount)
}

entry, err := NewLedgerEntry(entryID, w.id, transactionID, direction, amount, w.balance,
newBalance, now)
if err != nil {
    return LedgerEntry{}, err
}

w.balance = newBalance
w.version++
w.updatedAt = now
return entry, nil
```

Há um teste que executa oito movimentações inválidas e confere, em cada uma, que saldo, versão e data não mudaram.

### Criação e reidratação

| FUNÇÃO | QUANDO | O QUE FAZ |
|---|---|---|
| `OpenWallet` | Carteira nova | Versão 1, saldo inicial, as duas datas iguais |
| `RehydrateWallet` | Linha lida do banco | Restaura exatamente o que recebeu; só valida |

É a diferença entre um construtor público e o que um ORM faria por reflexão em C#. Aqui os dois caminhos são explícitos.

## 4. LedgerEntry

Um lançamento guarda carteira, transação, direção ( `DEBIT` ou `CREDIT` ), valor, saldo antes e saldo depois. O construtor refaz a conta e recusa qualquer combinação que não feche:

```
switch direction {
case LedgerDebit:
    expected, err = balanceBefore.Sub(amount)
case LedgerCredit:
    expected, err = balanceBefore.Add(amount)
default:
    return LedgerEntry{}, invalid("direction", "must be DEBIT or CREDIT")
}
...
if !expected.Equal(balanceAfter) {
    return LedgerEntry{}, invalid("balanceAfter", "does not match balanceBefore and amount")
}
```

Não existe método que altere um `LedgerEntry` . Para corrigir um erro financeiro, cria-se um novo lançamento no sentido contrário. É o que "append-only" significa.

## 5. WagerTransaction e a máquina de estados

| ESTADO | SIGNIFICADO | PODE IR PARA |
|---|---|---|
| `PENDING` | Aceita, ainda não concluída | `PENDING_REFERENCE` , `PROCESSED` , `REJECTED` , `FAILED` |
| `PENDING_REFERENCE` | Esperando a operação referenciada chegar | `PROCESSED` , `REJECTED` , `FAILED` |
| `PROCESSED` | Concluída com sucesso | Nenhum (terminal) |
| `REJECTED` | Recusada por regra de negócio | Nenhum (terminal) |
| `FAILED` | Falha permanente de infraestrutura | Nenhum (terminal) |

No código, a tabela acima é um mapa, e toda mudança de estado passa por uma única função:

```
var allowedTransitions = map[TransactionStatus][]TransactionStatus{
    StatusPending:          {StatusPendingReference, StatusProcessed, StatusRejected,
StatusFailed},
    StatusPendingReference: {StatusProcessed, StatusRejected, StatusFailed},
}

func (t *WagerTransaction) transition(to TransactionStatus, now time.Time) error {
    ...
    if !slices.Contains(allowedTransitions[t.status], to) {
        return fmt.Errorf("%w: %s -> %s", ErrInvalidTransition, t.status, to)
    }
    t.status = to
    t.updatedAt = now
    return nil
}
```

Os estados terminais não aparecem como chave do mapa. Buscar uma chave ausente devolve uma lista vazia, então toda transição a partir deles é recusada, sem precisar de um `if` para cada caso.

Os métodos públicos (`MarkProcessed`, `MarkRejected`, `MarkFailed`, `MarkPendingReference`) chamam `transition` e gravam o que acompanha cada resultado: o saldo observado ou o código de falha.

### Validação por tipo, na criação

| TIPO | VALOR | REFERÊNCIA |
|---|---|---|
| `BET` | Maior que zero | Não aceita |
| `WIN` | Maior que zero | Opcional |
| `LOSS` | Exatamente `0.00` | Não aceita |
| `REFUND` | Maior que zero | Obrigatória |
| `ROLLBACK` | Maior que zero | Obrigatória |
| `OPENING` | Recusado em `NewWagerTransaction`; só existe por `NewOpeningTransaction` | |

A abertura de carteira tem construtor próprio. Ela já nasce `PROCESSED` e não carrega provedor, ID externo, chave, hash, rodada nem jogo.

## 6. ApplyWager: as regras dos cinco tipos

```
func ApplyWager(tx *WagerTransaction, wallet *Wallet, ref Reference, entryID string, now
time.Time) (*LedgerEntry, error)
```

Recebe a transação, a carteira e o que foi encontrado sobre a referência. Decide o resultado e o grava nos dois objetos.

1. A carteira é do mesmo jogador? Se não: `PLAYER_MISMATCH`.
2. A moeda é a da carteira? Se não: `CURRENCY_MISMATCH`.
3. Tem referência? Confere contexto, tipo, estado, valor e se já foi revertida.

4. Referência ainda não chegou ou está pendente: a transação fica `PENDING_REFERENCE`.
5. Define o movimento: débito, crédito ou nenhum.
6. Movimenta a carteira. Falta de saldo vira rejeição.
7. Marca `PROCESSED` com o saldo resultante.

| TIPO | MOVIMENTO | REFERÊNCIA ACEITA |
|---|---|---|
| `BET` | Débito | — |
| `WIN` | Crédito | `BET` |
| `LOSS` | Nenhum; a carteira não é tocada | — |
| `REFUND` | Crédito | `BET` |
| `ROLLBACK` | De `BET`: crédito. De `WIN` ou `REFUND`: débito | `BET`, `WIN`, `REFUND` |

> **Rejeição não é erro.** Aposta sem saldo é um resultado de negócio esperado. `ApplyWager` retorna sem erro, e a transação fica `REJECTED` com um `failureCode`. Isso importa porque a rejeição precisa ser gravada no banco e devolvida igual em um reenvio. O retorno de erro fica para uso incorreto do código, como aplicar uma transação já terminal.

### Dois tipos de "não deu certo"

| | ENTRADA INVÁLIDA | REJEIÇÃO DE NEGÓCIO |
|---|---|---|
| Exemplo | `BET` com valor zero | `BET` sem saldo |
| Onde é detectada | No construtor | Em `ApplyWager` |
| Como aparece no código | Erro do tipo `*ValidationError` | Transação `REJECTED` + `failureCode` |
| É gravada? | Não | Sim |
| O provedor pode corrigir e reenviar? | Sim | Não; é definitivo |
| HTTP (spec 07) | 400 | 422 |

### Códigos de falha

| CÓDIGO | QUANDO |
|---|---|
| `INSUFFICIENT_FUNDS` | `BET` sem saldo |
| `REVERSAL_INSUFFICIENT_FUNDS` | `ROLLBACK` que debitaria mais que o saldo |
| `BALANCE_LIMIT_EXCEEDED` | Crédito que passaria do limite do `int64` |
| `PLAYER_MISMATCH` | A carteira é de outro jogador |
| `CURRENCY_MISMATCH` | Moeda diferente da carteira |
| `REFERENCE_NOT_FOUND` | A referência não chegou até expirar (spec 08) |
| `REFERENCE_NOT_PROCESSED` | A referência terminou rejeitada ou com falha |
| `REFERENCE_MISMATCH` | Diverge em provedor, jogador, carteira, moeda ou rodada |
| `REFERENCE_AMOUNT_MISMATCH` | Valor da reversão diferente do original |
| `REFERENCE_KIND_NOT_ALLOWED` | Por exemplo, `REFUND` de um `WIN` |
| `ALREADY_REVERSED` | A referência já tem uma reversão processada |

## 7. Conceitos de Go

### Tipo de erro próprio e `errors.As`

O erro sentinela diz *qual* erro aconteceu. Quando o erro precisa carregar dados, usa-se uma struct que implementa o método `Error()`:

```
type ValidationError struct {
    Field  string
    Reason string
}

func (e *ValidationError) Error() string {
    return fmt.Sprintf("invalid %s: %s", e.Field, e.Reason)
}
```

Quem recebe usa `errors.As` para recuperar a struct e ler os campos. É o `catch (ValidationException ex)` seguido de `ex.Field`:

```
var validation *ValidationError
if errors.As(err, &validation) {
    // validation.Field, validation.Reason
}
```

| | `errors.Is` | `errors.As` |
|---|---|---|
| Pergunta | É este erro específico? | É um erro deste tipo? |
| Compara com | Uma variável ( `ErrInsufficientFunds` ) | Um tipo ( `*ValidationError` ) |
| Dá acesso a dados | Não | Sim |

Não há `implements` : `ValidationError` vale como `error` só por ter o método `Error() string` . É a interface implícita do Go.

### Enum como tipo nomeado

Go não tem `enum` . O idioma é criar um tipo a partir de `string` e declarar as constantes dele:

```
type TransactionKind string

const (
    KindOpening  TransactionKind = "OPENING"
    KindBet      TransactionKind = "BET"
    ...
)

func (k TransactionKind) IsReversal() bool {
    return k == KindRefund || k == KindRollback
}
```

O compilador impede passar uma `string` comum onde se espera `TransactionKind` , e o tipo pode ter métodos. Mas ele não impede um valor desconhecido como `TransactionKind("DEPOSIT")` , por isso os construtores validam.

### Ponteiro ou valor

| TIPO | RECEIVER | MOTIVO |
|---|---|---|
| `Money` , `LedgerEntry` | Valor: `(m Money)` | Imutáveis; copiar é seguro e barato |
| `Wallet` , `WagerTransaction` | Ponteiro: `(w *Wallet)` | Têm identidade e mudam de estado; todos precisam ver o mesmo objeto |

É a distinção entre `struct` e `class` do C#, só que escolhida em cada método.

### Struct de parâmetros

Go não tem argumentos nomeados nem opcionais. Para uma função com muitos parâmetros do mesmo tipo, agrupa-se tudo em uma struct, o que evita trocar a ordem por engano:

```
tx, err := domain.NewWagerTransaction(domain.NewWagerTransactionParams{
    ID:         "...",
    ProviderID: "provider-a",
    Kind:       domain.KindBet,
    Money:      money,
    Now:        now,
})
```

### Outros pontos

- `switch` **sem expressão** funciona como uma cadeia de `if / else if` e não precisa de `break`.
- **Closure:** em `ApplyWager`, `reject` é uma função declarada dentro de outra, que enxerga `tx`, `wallet` e `now`.
- **Retorno `(valor, bool)`:** `ResultBalance()` devolve o saldo e se ele existe. É o padrão do Go para "pode não ter valor", no lugar de `null`.
- `time.Time{}` é o valor zero de data; `IsZero()` detecta data não informada.

## 8. Decisões

| DECISÃO | MOTIVO |
|---|---|
| IDs e o instante entram como parâmetro | O domínio não gera UUID nem lê o relógio: testes determinísticos e nenhuma dependência externa |
| `Debit` e `Credit` devolvem o lançamento | Saldo e ledger nascem juntos; não há como mudar um sem o outro |
| Rejeição de negócio é resultado, não erro | Precisa ser persistida e reproduzida em reenvios |
| Uma transação aceita uma única reversão processada, de qualquer tipo | Impede devolver duas vezes o mesmo débito (`REFUND` + `ROLLBACK` da mesma aposta) |
| `ROLLBACK` de `ROLLBACK` é recusado | O desafio lista só `BET`, `WIN` e `REFUND` como alvos |
| `WIN` com referência espera a aposta chegar | Mesma regra das reversões; um caminho só para referência ausente |
| Código próprio para valor divergente (`REFERENCE_AMOUNT_MISMATCH`) | É o erro mais provável de integração; fica fácil de diagnosticar |
| `LedgerEntry` tem um construtor só | Não há comportamento a reaplicar na reidratação, apenas a equação a conferir |

> **O que esta spec ainda não garante.** As regras estão corretas para um processo só, uma operação por vez. Duas apostas *simultâneas*, em instâncias diferentes, carregariam cada uma a sua cópia da carteira com saldo 100. Quem resolve isso é o lock no banco (spec 05) e as constraints (spec 04). O domínio decide; o banco garante.

## 9. Como foi verificado

| VERIFICAÇÃO | RESULTADO |
|---|---|
| `go test ./internal/domain/ -cover` | 243 casos passam; 99,7% das instruções |
| `go test -race ./...` (em container) | Passou |
| `gofmt -l .` e `go vet ./...` | Sem saída |
| Imports do pacote `domain` | Só biblioteca padrão |

### O que os testes cobrem

- **Carteira:** abertura, reidratação, débito, crédito, e oito movimentações inválidas que não podem alterar o estado.
- **Ledger:** a equação do saldo nas duas direções e onze entradas inválidas.
- **Máquina de estados:** as 20 combinações de 5 estados de origem por 4 destinos.
- **Política por tipo:** 15 combinações de tipo e valor, 10 de tipo e referência.
- **Regras:** cada tipo no caminho feliz, as duas apostas de 80 sobre 100, a reversão que chega antes da referência e 10 rejeições por referência.

## 10. Perguntas de revisão

### Por que `Debit` devolve um `LedgerEntry` em vez de só alterar o saldo?

Para que seja impossível mudar o saldo sem produzir o lançamento. O desafio exige que cada mudança financeira tenha o registro correspondente.

### Uma aposta sem saldo faz `ApplyWager` devolver erro?

Não. A função retorna sem erro e a transação fica `REJECTED` com `INSUFFICIENT_FUNDS`. É um resultado de negócio que será gravado.

### Qual a diferença entre `errors.Is` e `errors.As`?

`Is` compara com um erro específico (uma variável). `As` procura um erro de um tipo e devolve a struct, dando acesso aos campos.

### Por que existem `OpenWallet` e `RehydrateWallet`?

Criar define o estado inicial (versão 1). Reidratar restaura o que estava gravado, sem aplicar nenhuma regra de criação.

### Como o código impede uma transição a partir de `PROCESSED`?

`PROCESSED` não é chave no mapa de transições. A busca devolve lista vazia e `transition` retorna `ErrInvalidTransition`.

### Um `ROLLBACK` debita ou credita?

Depende do original: faz o movimento contrário. De uma `BET` (débito), credita. De um `WIN` ou `REFUND` (créditos), debita.

### O domínio sozinho impede saldo negativo com duas instâncias rodando?

Não. Cada instância teria a sua cópia da carteira em memória. A garantia entre processos vem do lock e das constraints do banco.

### Por que `LOSS` não altera a versão da carteira?

A versão só sobe quando o saldo muda. `LOSS` não movimenta nada, então `ApplyWager` nem chama a carteira.

## Ordem de leitura do código

1. `errors.go`
2. `ledger.go`
3. `wallet.go` e `wallet_test.go`
4. `wager_transaction.go`
5. `wager_rules.go` e `wager_rules_test.go`

SPEC 04

# Schema e migrations

As mesmas regras do domínio, agora escritas no banco. Se a aplicação tiver um bug, ou se duas instâncias disputarem a mesma carteira, o PostgreSQL recusa o que quebraria uma invariante.

## 1. O que a spec pedia

- Migrations versionadas, com aplicação e reversão.
- Saldo nunca negativo, garantido pelo banco.
- Unicidade da carteira por jogador e moeda, da operação externa e da chave de idempotência.
- Ledger com um lançamento por transação e carteira, protegido contra edição e exclusão.
- Schema que distingue operação interna ( `OPENING` ) de externa e impede crédito inicial duplicado.
- Tabelas de inbox e outbox.

## 2. O que foi construído

| ARQUIVO | CONTEÚDO |
|---|---|
| `migrations/000001_create_wallets` | Tabela `wallets` |
| `migrations/000002_create_wager_transactions` | Tabela `wager_transactions` e seus índices únicos parciais |
| `migrations/000003_create_wallet_ledger_entries` | Tabela do ledger e os triggers que a tornam append-only |
| `migrations/000004_create_inbox_and_outbox` | Tabelas `inbox_messages` e `outbox_events` |
| `migrations/migrations_test.go` | Teste que aplica as migrations e tenta violar cada invariante |
| `docker-compose.yml` | Serviço `migrate` , que roda antes da aplicação |

Cada migration tem dois arquivos: `.up.sql` (aplica) e `.down.sql` (desfaz). O número no início do nome define a ordem.

## 3. Como as migrations funcionam

A ferramenta é o `golang-migrate` . Ela cria no banco a tabela `schema_migrations` , com uma linha: a versão atual. `up` aplica os arquivos com número maior que essa versão; `down` roda os `.down.sql` na ordem inversa.

É o mesmo papel das migrations do Entity Framework ou do TypeORM, com uma diferença: aqui o SQL é escrito à mão, não gerado a partir de classes.

1. `docker compose up` sobe o Postgres e espera o healthcheck.
2. O serviço `migrate` aplica o que falta e termina.

3. O serviço `app` só inicia depois que o `migrate` termina com sucesso (`service_completed_successfully`).

| COMANDO | EFEITO |
|---|---|
| `docker compose run --rm migrate` | Aplica tudo o que falta |
| `... migrate -path=/migrations -database=<url> down 1` | Reverte a última migration |
| `... down -all` | Reverte todas |
| `... version` | Mostra a versão atual |

## 4. As tabelas

| TABELA | UMA LINHA É | PODE MUDAR? |
|---|---|---|
| `wallets` | Uma carteira e seu saldo atual | Sim: saldo, versão e `updated_at` |
| `wager_transactions` | Uma operação, interna ou externa | Sim: estado, código de falha, saldo resultante |
| `wallet_ledger_entries` | Um débito ou crédito | Nunca |
| `inbox_messages` | Uma mensagem SQS já recebida por um consumidor | Só a conclusão |
| `outbox_events` | Um evento a publicar | Só os campos de envio |

Dinheiro é sempre um par de colunas: `BIGINT` com os centavos e `CHAR(3)` com a moeda. É a mesma representação do `Money` da spec 02, então gravar e ler não converte nada.

## 5. Os quatro mecanismos usados

### CHECK: regra sobre uma linha

```
CONSTRAINT wallets_balance_non_negative CHECK (balance_minor >= 0)
```

O banco avalia a expressão em todo `INSERT` e `UPDATE`. Se der falso, o comando falha com SQLSTATE `23514`. Esta é a rede de segurança contra saldo negativo: mesmo que o lock da aplicação falhasse, o `UPDATE` não passaria.

O `CHECK` enxerga só a própria linha. Ele serve para regras como "`LOSS` tem valor zero" ou "transação rejeitada tem código de falha", mas não para "só existe uma abertura por carteira".

```
CONSTRAINT wallet_ledger_entries_balance_equation CHECK (
    (direction = 'CREDIT' AND balance_after_minor = balance_before_minor + amount_minor)
    OR (direction = 'DEBIT' AND balance_after_minor = balance_before_minor - amount_minor)
)
```

É a mesma equação que `NewLedgerEntry` valida no domínio. A regra existe nos dois lugares de propósito.

### UNIQUE e índice único parcial: regra entre linhas

```
CONSTRAINT wager_transactions_external_key UNIQUE (provider_id, external_transaction_id)
```

Duas linhas com o mesmo par falham com SQLSTATE `23505`. É isso que impede a mesma operação de ser aplicada duas vezes quando duas instâncias a recebem ao mesmo tempo: uma insere, a outra recebe o erro.

Um índice único **parcial** aplica a unicidade só às linhas que passam em um filtro:

```
CREATE UNIQUE INDEX wager_transactions_one_opening_per_wallet
    ON wager_transactions (wallet_id)
    WHERE kind = 'OPENING';

CREATE UNIQUE INDEX wager_transactions_one_processed_reversal
    ON wager_transactions (reference_transaction_id)
    WHERE status = 'PROCESSED' AND kind IN ('REFUND', 'ROLLBACK');
```

O primeiro permite muitas transações por carteira, mas só uma `OPENING`. O segundo permite várias reversões rejeitadas da mesma aposta, mas só uma processada. É a regra "uma reversão por transação" da spec 03, agora à prova de concorrência.

> **NULL em UNIQUE.** O Postgres trata cada `NULL` como diferente dos outros. Por isso as transações `OPENING`, que têm `provider_id` nulo, não colidem entre si na constraint `(provider_id, external_transaction_id)`.

### Chave estrangeira composta: coerência entre tabelas

```
FOREIGN KEY (wallet_id, currency)       REFERENCES wallets (id, currency),
FOREIGN KEY (transaction_id, wallet_id) REFERENCES wager_transactions (id, wallet_id)
```

Uma FK comum garante que a carteira existe. A composta garante mais: que o lançamento está na **moeda da carteira** e pertence à **mesma carteira da transação**. Para isso a tabela de destino precisa de um `UNIQUE` nas mesmas colunas, e é por isso que `wallets` tem `UNIQUE (id, currency)`, que parece redundante.

### Trigger: o ledger append-only

```
CREATE FUNCTION reject_wallet_ledger_entry_change() RETURNS trigger
    LANGUAGE plpgsql AS
$$
BEGIN
    RAISE EXCEPTION 'wallet_ledger_entries is append-only: % is not allowed', TG_OP
        USING ERRCODE = 'restrict_violation';
END;
$$;

CREATE TRIGGER wallet_ledger_entries_append_only
    BEFORE UPDATE OR DELETE ON wallet_ledger_entries
    FOR EACH ROW EXECUTE FUNCTION reject_wallet_ledger_entry_change();
```

Um trigger é uma função que o banco executa quando algo acontece na tabela. Esta roda antes de cada `UPDATE` ou `DELETE` e sempre lança erro; `TG_OP` é o nome da operação tentada. Só `INSERT` passa.

Há um segundo trigger para `TRUNCATE`, porque o trigger por linha não dispara nesse comando.

## 6. Interno e externo na mesma tabela

`OPENING` não tem provedor, ID externo, chave, hash, rodada nem jogo. As outras operações têm todos. As colunas são anuláveis e um `CHECK` exige um dos dois formatos, sem meio-termo:

```
CONSTRAINT wager_transactions_origin CHECK (
    (kind = 'OPENING' AND status = 'PROCESSED'
        AND num_nonnulls(provider_id, external_transaction_id, ...) = 0)
    OR
    (kind <> 'OPENING'
        AND num_nulls(provider_id, external_transaction_id, ...) = 0)
)
```

`num_nulls` e `num_nonnulls` contam quantos argumentos são nulos ou não nulos. Evitam uma fila de `IS NULL AND ... IS NULL`.

## 7. Domínio e banco: quem garante o quê

| REGRA | NO DOMÍNIO | NO BANCO |
|---|---|---|
| Saldo não negativo | `Wallet.Debit` | `CHECK` |
| Equação do lançamento | `NewLedgerEntry` | `CHECK` |
| Valor e referência por tipo | `NewWagerTransaction` | `CHECK` |
| Uma reversão processada | `Reference.Reversed` | Índice único parcial |
| Operação externa única | — | `UNIQUE` |
| Uma carteira por jogador e moeda | — | `UNIQUE` |
| Ledger imutável | Struct sem setters | Trigger |

> **Por que repetir no banco.** Com três instâncias, cada uma tem a sua cópia dos objetos em memória. A validação do domínio só enxerga essa cópia. O banco é o único lugar que todas as instâncias compartilham, então é o único que pode dizer "isso já existe".

O domínio continua necessário: ele dá a mensagem de erro certa e o `failureCode` antes de chegar ao banco. A constraint cobre o que escapar.

## 8. Decisões

| DECISÃO | MOTIVO |
|---|---|
| `kind`, `status` e `direction` em `TEXT` + `CHECK`, não `ENUM` | Mudar a lista de um `CHECK` é reversível; remover um valor de `ENUM` do Postgres não é |
| IDs internos em `UUID`, identificadores de fora em `TEXT` | O formato de `providerId` ou `externalTransactionId` é do provedor |
| Coluna `sequence` (identity) no ledger | Ordem estável para a paginação. `created_at` pode empatar; a sequência não |
| Sem `DEFAULT now()` | O instante vem da aplicação, o mesmo que o domínio usou |
| Constraints com nome explícito | O nome vem no erro do Postgres, o que permite à spec 05 traduzir cada violação em um erro de aplicação |
| `locked_until` na outbox | Um publisher reserva o evento por um prazo; se morrer, outro assume (spec 10) |
| Quatro migrations em vez de uma | Cada arquivo trata de um assunto e pode ser revertido sozinho |

## 9. O teste

Constraint não se testa com mock: o teste precisa de um PostgreSQL real. Ele lê a URL de `TEST_DATABASE_URL` e é pulado quando ela não existe.

```
func connect(t *testing.T) *pgx.Conn {
    url := os.Getenv("TEST_DATABASE_URL")
    if url == "" {
        t.Skip("TEST_DATABASE_URL is not set")
    }
    // cria um schema descartável e aponta o search_path para ele
    t.Cleanup(func() { /* DROP SCHEMA ... CASCADE */ })
}
```

- `t.Skip` marca o teste como pulado, não como falho.
- `t.Cleanup` registra uma função para rodar ao fim do teste. É o `afterEach` do Jest ou o `Dispose` do xUnit.
- Cada teste cria um **schema** próprio (um namespace de tabelas). Assim ele não interfere nas tabelas reais nem em outro teste.

Cada violação é conferida pelo código e pelo nome da constraint, não só por "deu erro":

```
var pgErr *pgconn.PgError
if !errors.As(err, &pgErr) { ... }
// pgErr.Code           == "23514"
// pgErr.ConstraintName == "wallets_balance_non_negative"
```

É o `errors.As` da spec 03 em uso real: extrai do erro a struct do driver, com o SQLSTATE dentro.

| SQLSTATE | SIGNIFICADO |
|---|---|
| `23505` | Violação de `UNIQUE` |
| `23514` | Violação de `CHECK` |
| `23503` | Violação de chave estrangeira |
| `23001` | `restrict_violation`: o código escolhido para o trigger do ledger |

## 10. Como foi verificado

| VERIFICAÇÃO | RESULTADO |
|---|---|
| `migrate up` → `down -all` → `up` | Depois do `down` sobra só `schema_migrations`; o segundo `up` aplica as 4 |
| `go test ./migrations/` com banco real | 22 violações recusadas e 5 casos válidos aceitos |
| `go test -race ./...` (em container) | Passou |
| `docker compose up --build` | `migrate` termina com código 0, depois `app` sobe e `/health/ready` responde 200 |

```
docker compose up -d postgres
docker compose run --rm migrate
TEST_DATABASE_URL='postgres://wagering:wagering@localhost:5432/wagering?sslmode=disable' go test
./migrations/
```

## 11. Perguntas de revisão

**Se o domínio já impede saldo negativo, para que serve o `CHECK`?**

O domínio valida a cópia em memória de uma instância. O `CHECK` vale para todas as instâncias e para qualquer bug da aplicação.

**Qual a diferença entre `UNIQUE` e índice único parcial?**

O parcial tem um `WHERE`: a unicidade vale só para as linhas que passam no filtro. Ex.: uma única `OPENING` por carteira.

**Por que duas `OPENING` não colidem em `UNIQUE (provider_id, external_transaction_id)`?**

Os dois campos são `NULL` nelas, e o Postgres não considera dois `NULL` iguais.

**Como o ledger fica imutável?**

Um trigger `BEFORE UPDATE OR DELETE` lança erro em toda tentativa, e outro faz o mesmo para `TRUNCATE`.

**O que o `CHECK` não consegue fazer?**

Comparar com outras linhas ou tabelas. Para isso existem `UNIQUE`, chave estrangeira e trigger.

**Para que serve `UNIQUE (id, currency)` em `wallets`, se `id` já é único?**

Para ser o destino da FK composta do ledger, que garante lançamento na moeda da carteira.

**Como o `app` sabe que o schema está pronto?**

No Compose ele depende do `migrate` com `service_completed_successfully`.

**Por que o teste de schema é pulado no `go test ./...` comum?**

Sem `TEST_DATABASE_URL` ele chama `t.Skip`. Ele precisa de um Postgres real.

### Ordem de leitura

1. `000001_create_wallets.up.sql`
2. `000002_create_wager_transactions.up.sql`
3. `000003_create_wallet_ledger_entries.up.sql`
4. `000004_create_inbox_and_outbox.up.sql`
5. `migrations_test.go`: a lista `rejected` é um resumo executável das invariantes

SPEC 05

# Persistência e transação SQL

A ponte entre o domínio e o banco: repositórios com SQL escrito à mão, uma transação compartilhada por todos eles e o lock por carteira que impede duas instâncias de gastarem o mesmo saldo.

## 1. O que a spec pedia

- `pgx` com SQL explícito; transações, locks e constraints visíveis no código.
- Um único mecanismo que delimita a transação SQL, compartilhada por todos os repositórios de um caso de uso.
- Lock por carteira. Lock global é proibido; carteiras diferentes avançam em paralelo.
- Nenhuma atualização confirmada pode ser perdida.
- Todo I/O recebe `context.Context` e respeita cancelamento.
- Falha transitória distinguível de falha permanente.

## 2. O que foi construído

| ARQUIVO | CONTEÚDO |
|---|---|
| `internal/application/ports.go` | As interfaces dos repositórios e os erros que os casos de uso enxergam |
| `internal/infra/postgres/tx.go` | `TxManager` : abre, confirma ou desfaz a transação |
| `internal/infra/postgres/errors.go` | Traduz erro do PostgreSQL para erro de aplicação |
| `wallet_repository.go` | Carteira: inserir, ler, ler com lock, atualizar saldo |
| `wager_transaction_repository.go` | Transação: inserir, atualizar, buscar por ID, por ID externo e por chave |
| `ledger_repository.go` , `outbox_repository.go` | Só inserção |
| `internal/testdb/testdb.go` | Banco de teste: um schema migrado por teste |

## 3. Ports: a interface fica com quem usa

```
// internal/application/ports.go
type WalletRepository interface {
    Insert(ctx context.Context, wallet *domain.Wallet) error
    Get(ctx context.Context, id string) (*domain.Wallet, error)
    GetForUpdate(ctx context.Context, id string) (*domain.Wallet, error)
    UpdateBalance(ctx context.Context, wallet *domain.Wallet) error
}
```

A interface está no pacote `application`, que é quem **consome** o repositório. A implementação está em `infra/postgres`. Assim `application` não importa nada de banco, e a regra de dependência do desafio se mantém.

Em Go a implementação não declara que implementa. `*postgres.WalletRepository` satisfaz a interface só por ter os mesmos métodos. Em C# seria `class WalletRepository : IWalletRepository`; aqui não existe essa linha.

Quem liga as duas pontas é o Fx:

```
fx.Annotate(NewWalletRepository, fx.As(new(application.WalletRepository)))
```

`fx.As` registra o resultado do construtor **como a interface**. É o `services.AddScoped<IWalletRepository, WalletRepository>()` do C#. Um caso de uso vai pedir `application.WalletRepository` e receber a implementação do Postgres sem conhecê-la.

## 4. A transação viaja no context

```
func (m *TxManager) WithinTx(ctx context.Context, fn func(ctx context.Context) error) error {
    if _, ok := ctx.Value(txContextKey{}).(pgx.Tx); ok {
        return fn(ctx)                      // já existe transação: entra nela
    }

    tx, err := m.pool.Begin(ctx)
    if err != nil {
        return translate(err)
    }
    defer func() { _ = tx.Rollback(context.WithoutCancel(ctx)) }()

    if err := fn(context.WithValue(ctx, txContextKey{}, tx)); err != nil {
        return err                          // o defer faz o rollback
    }
    return translate(tx.Commit(ctx))
}
```

1. `Begin` pega uma conexão do pool e abre a transação.
2. `context.WithValue` cria um novo `ctx` que carrega a transação.
3. A função do caso de uso recebe esse `ctx` e o repassa a cada repositório.
4. Cada repositório procura a transação no `ctx`. Achou: usa ela. Não achou: usa o pool.
5. Retornou `nil`: commit. Retornou erro: rollback.

```
func querierFrom(ctx context.Context, pool *pgxpool.Pool) querier {
    if tx, ok := ctx.Value(txContextKey{}).(pgx.Tx); ok {
        return tx
    }
    return pool
}
```

O caso de uso não vê `pgx.Tx` em lugar nenhum. Ele só escreve:

```
err := txManager.WithinTx(ctx, func(ctx context.Context) error {
    wallet, err := wallets.GetForUpdate(ctx, walletID)
    // ... regra do domínio ...
    if err := wagers.Insert(ctx, tx); err != nil { return err }
    if err := wallets.UpdateBalance(ctx, wallet); err != nil { return err }
    return ledger.Insert(ctx, *entry)
})
```

É o papel do `TransactionScope` do C# ou do `@Transactional`, feito com um parâmetro explícito em vez de estado escondido na thread.

### Detalhes do WithinTx

| TRECHO | POR QUÊ |
|---|---|
| `txContextKey struct{}` | A chave do `context` é um tipo privado do pacote. Nenhum outro pacote consegue ler ou sobrescrever o valor |
| `ctx.Value(...).(pgx.Tx)` | Type assertion: `Value` devolve `any`; a forma com `, ok` não dá panic se o valor não existir |
| `defer tx.Rollback(...)` | Roda em qualquer saída, inclusive panic. Depois de um commit não faz nada |
| `context.WithoutCancel(ctx)` | Se o `ctx` foi cancelado, o rollback com ele também falharia. Este mantém os valores e remove o cancelamento |
| Interface `querier` | Pool e transação têm os mesmos métodos `Exec`, `Query` e `QueryRow`. O repositório depende só deles |

## 5. O lock por carteira

```
SELECT id::text, player_id, currency, balance_minor, version, created_at, updated_at
  FROM wallets
 WHERE id = $1
   FOR UPDATE
```

`FOR UPDATE` trava **a linha** até o fim da transação. Outra transação que pedir a mesma linha com `FOR UPDATE` fica parada esperando. Quando a primeira termina, a segunda lê o saldo já atualizado.

| INSTANTE | INSTÂNCIA A (APOSTA DE 80) | INSTÂNCIA B (APOSTA DE 80) |
|---|---|---|
| 1 | `GetForUpdate`: lê saldo 100 | |
| 2 | | `GetForUpdate`: **espera** |
| 3 | Debita, grava saldo 20, commit | |
| 4 | | Lê saldo **20** |
| 5 | | Domínio rejeita: `INSUFFICIENT_FUNDS` |

Sem o lock, as duas leriam 100 e as duas gravariam 20: uma aposta sumiria do saldo. Esse é o *lost update*.

O lock é da linha, não da tabela. Uma aposta em outra carteira não espera ninguém.

### Duas proteções extras

```
UPDATE wallets
   SET balance_minor = $2, version = $3, updated_at = $4
 WHERE id = $1 AND version = $3 - 1
```

- **Guarda de versão.** O `UPDATE` só acerta a linha se a versão no banco for a anterior à nova. Se alguém gravou sem o lock, nenhuma linha é afetada e o repositório devolve `ErrConcurrentUpdate`.
- `GetForUpdate` **fora de transação retorna erro.** Sem transação o lock seria solto no mesmo instante.

Com o `CHECK (balance_minor >= 0)` da spec 04, são três camadas contra saldo errado: lock, guarda de versão e constraint.

## 6. SQL e mapeamento

Sem ORM. Cada repositório escreve o SQL e converte as colunas na mão.

```
err := row.Scan(&id, &playerID, &currency, &balanceMinor, &version, &createdAt, &updatedAt)
if errors.Is(err, pgx.ErrNoRows) {
    return nil, application.ErrWalletNotFound
}
balance, err := domain.MoneyFromMinorUnits(balanceMinor, currency)
return domain.RehydrateWallet(id, playerID, balance, version, createdAt.UTC(), updatedAt.UTC())
```

- `Scan` recebe ponteiros ( `&id` ) e preenche as variáveis, na ordem das colunas do `SELECT`.
- A leitura termina em `RehydrateWallet`, a função de reidratação da spec 03: restaura o estado sem aplicar regra de criação.
- Parâmetros são `$1`, `$2` ..., nunca concatenação de string. É o que evita SQL injection.

| NO DOMÍNIO | NO BANCO | CONVERSÃO |
|---|---|---|
| `Money` | `BIGINT` + `CHAR(3)` | `MinorUnits()` / `MoneyFromMinorUnits` |
| Campo opcional vazio ( `""` ) | `NULL` | `NULLIF($8, '')` na escrita, `COALESCE(col, '')` na leitura |
| Saldo resultante não observado | `NULL` | Ponteiro `*int64` : `nil` vira `NULL` |
| `string` com UUID | `UUID` | `id::text` na leitura |

> **Ponteiro como "pode ser nulo".** `int64` não tem valor nulo: o zero é `0`. Para representar ausência usa-se `*int64`, que pode ser `nil`. É o `long?` do C#.

## 7. Tradução de erros

O caso de uso não pode depender de códigos do PostgreSQL. `translate` converte cada erro em um erro de `application`:

| DO POSTGRESQL | VIRA | TIPO |
|---|---|---|
| `23505` em `wallets_player_currency_key` | `ErrWalletAlreadyExists` | Permanente |
| `23505` em `wager_transactions_external_key` | `ErrDuplicateExternalTransaction` | Permanente |
| `23505` em `wager_transactions_idempotency_key` | `ErrDuplicateIdempotencyKey` | Permanente |
| Outras da classe `23` | `ErrInvariantViolation` | Permanente (é bug) |
| `40001`, `40P01`, classes `08`, `53`, `57`, erro de rede | `ErrUnavailable` | **Transitório** |

É aqui que os nomes explícitos das constraints da spec 04 são usados: o mesmo código `23505` vira erros diferentes conforme a constraint violada.

```
return fmt.Errorf("%w: %w", application.ErrDuplicateIdempotencyKey, pgErr)
```

Dois `%w` encadeiam os dois erros. `errors.Is(err, application.ErrDuplicateIdempotencyKey)` é verdadeiro, e o erro original do Postgres continua acessível para o log.

> **Transitório × permanente.** Transitório: repetir a mesma chamada mais tarde pode funcionar (banco reiniciando, deadlock). Vai virar `503` no HTTP e reentrega no SQS. Permanente: repetir dá o mesmo resultado. Essa distinção é exigida pelo desafio e decide o que vai para a DLQ.

Depois de uma violação de constraint, o PostgreSQL marca a transação como abortada: qualquer comando seguinte falha. Por isso o caso de uso retorna o erro em vez de tentar continuar.

## 8. Testes

O pacote `testdb` cria um schema novo para cada teste, aplica as migrations nele e apaga tudo no fim. Os testes rodam em paralelo com o banco de desenvolvimento sem interferir.

```
config.ConnConfig.RuntimeParams["search_path"] = schema
```

Toda conexão do pool de teste já nasce apontando para o schema descartável.

### O teste do lock

```
locked := make(chan struct{})
release := make(chan struct{})
go func() {
    holderDone <- f.tx.WithinTx(ctx, func(ctx context.Context) error {
        f.wallets.GetForUpdate(ctx, id(1))
        close(locked)      // avisa: estou com o lock
        <-release          // segura até mandarem soltar
        return nil
    })
}()
<-locked
```

- `go func() { ... }()` inicia uma goroutine: a função roda em paralelo.
- Um **channel** é um canal de comunicação entre goroutines. `<-locked` bloqueia até alguém enviar ou fechar o canal.
- `close(locked)` libera todos que estão esperando. Canais de `struct{}` servem só como sinal, sem dado.

Com o lock preso, o teste tenta travar a mesma carteira com prazo de 300 ms e espera `context.DeadlineExceeded`. Isso prova duas coisas de uma vez: a segunda transação espera, e o cancelamento do `context` interrompe a query.

### O teste de concorrência

```
var wg sync.WaitGroup
for n := range bets {
    wg.Go(func() { /* uma aposta de 1.00 */ })
}
wg.Wait()
```

`sync.WaitGroup` espera um grupo de goroutines terminar: é o `Task.WhenAll`. Vinte apostas de 1.00 disputam uma carteira de 20.00.

## 9. Como foi verificado

| VERIFICAÇÃO | RESULTADO |
|---|---|
| Erro dentro de `WithinTx` | Carteira, transação, ledger e outbox ficam com 0 linhas; com sucesso, 1 em cada |
| Mesma carteira travada | Segunda transação espera e é cortada pelo prazo; outra carteira trava sem esperar |
| 20 apostas simultâneas de 1.00 sobre 20.00 | Saldo 0.00, versão 21, 20 lançamentos; a 21ª é rejeitada |
| Gravação sem lock | `ErrConcurrentUpdate` |
| Ida e volta de `Money` | Valor e moeda idênticos, inclusive no maior saldo representável |
| `go test -race` (em container, 3 repetições) | Passou |
| Cobertura do pacote `postgres` | 83,6% |

O teste de concorrência roda dentro de **um** processo, com várias conexões. A prova com três processos independentes, exigida pelo desafio, fica na spec 15.

## 10. Uma correção no schema

Ao escrever o repositório apareceu um furo da spec 04: o saldo resultante tinha só a coluna de valor. Em uma rejeição por `CURRENCY_MISMATCH`, o saldo devolvido ao provedor está na moeda da carteira, que é diferente da moeda da transação. Sem a moeda gravada, um replay devolveria o saldo com a moeda errada.

A coluna `result_balance_currency` foi acrescentada, com um `CHECK` que exige as duas colunas juntas. O teste de ida e volta cobre exatamente esse caso.

## 11. Perguntas de revisão

**Como um repositório sabe que está dentro de uma transação?**

Ele procura a transação no `context`. O `TxManager` a colocou lá com `context.WithValue`.

**Por que a interface `WalletRepository` fica em `application` e não em `postgres`?**

Porque quem usa define o contrato. Assim `application` não importa nada de banco.

**O que `FOR UPDATE` trava, e até quando?**

As linhas retornadas pelo `SELECT`, até o commit ou rollback da transação.

**O que é lost update e como é evitado aqui?**

Duas transações leem o mesmo saldo e uma sobrescreve a outra. O lock faz a segunda ler só depois do commit da primeira.

**Para que serve a guarda de versão se já existe o lock?**

Para transformar um uso errado (gravar sem lock) em erro, em vez de corromper o saldo em silêncio.

**Por que o rollback usa `context.WithoutCancel`?**

Se o `ctx` original foi cancelado, o rollback feito com ele também seria cancelado.

**Qual a diferença entre `ErrUnavailable` e `ErrInvariantViolation`?**

O primeiro é transitório: vale tentar de novo. O segundo é permanente: o banco recusou algo que o domínio não deveria ter produzido.

**O que `fx.As` faz?**

Registra o resultado de um construtor como uma interface, para que os consumidores peçam a interface e não o tipo concreto.

### Ordem de leitura do código

1. `internal/application/ports.go`
2. `internal/infra/postgres/tx.go`
3. `wallet_repository.go`
4. `errors.go` e `errors_test.go`
5. `wager_transaction_repository.go`
6. `repositories_test.go`: comece por `applyBet`, que é o caminho de escrita completo de uma aposta

SPEC 06

# Carteira: abertura, leitura e ledger paginado

O primeiro fluxo de ponta a ponta. Uma requisição HTTP atravessa todas as camadas construídas até aqui: handler, caso de uso, domínio, repositórios e banco.

## 1. O que a spec pedia

- `POST /wallets`, `GET /wallets/:id` e `GET /wallets/:id/ledger`.
- Abertura com saldo positivo grava, no mesmo commit: carteira, transação `OPENING`, lançamento de crédito e dois eventos na outbox.
- Abertura com saldo zero grava só a carteira.
- Segunda carteira para o mesmo jogador e moeda: conflito.
- Ledger paginado com cursor opaco e ordenação estável.

## 2. O que foi construído

| ARQUIVO | CONTEÚDO |
|---|---|
| `internal/domain/events.go` | Os quatro eventos do desafio e o envelope comum |
| `internal/application/open_wallet.go` | Caso de uso de abertura |
| `internal/application/get_wallet.go`, `list_ledger.go` | Leitura da carteira e paginação do ledger |
| `internal/infra/httpapi/wallets.go` | Os três handlers e os contratos JSON |
| `internal/infra/httpapi/errors.go` | Tradução de erro para status HTTP |
| `internal/infra/httpapi/middleware.go` | Correlation ID e recuperação de panic |
| `internal/infra/system/system.go` | Relógio e gerador de UUID v7 |

## 3. O caminho de uma requisição

1. **Middleware** lê ou gera o `X-Correlation-Id` e o coloca no `context`.
2. **Handler** decodifica o JSON e chama o caso de uso. Não tem regra de negócio.
3. **Caso de uso** cria os objetos de domínio e abre a transação SQL.
4. **Domínio** valida: `OpenWallet`, `NewOpeningTransaction`, `NewLedgerEntry`, eventos.
5. **Repositórios** gravam tudo dentro da mesma transação.
6. **Handler** converte o resultado, ou o erro, em resposta HTTP.

Cada camada conhece só a de dentro. O handler não sabe que existe Postgres; o caso de uso não sabe que existe HTTP. É por isso que o mesmo caso de uso vai servir ao consumidor SQS na spec 11.

## 4. O caso de uso de abertura

```
func (uc *OpenWallet) Execute(ctx context.Context, in OpenWalletInput) (*domain.Wallet, error) {
    now := uc.clock.Now()
    wallet, err := domain.OpenWallet(uc.ids.NewID(), in.PlayerID, in.InitialBalance, now)
    if err != nil {
        return nil, err
    }

    err = uc.tx.WithinTx(ctx, func(ctx context.Context) error {
        if err := uc.wallets.Insert(ctx, wallet); err != nil {
            return err
        }
        if wallet.Balance().IsZero() {
            return nil
        }
        return uc.recordOpeningCredit(ctx, wallet, in.CorrelationID)
    })
    ...
}
```

- A validação do domínio acontece **antes** de abrir a transação: entrada inválida não gasta conexão.
- Saldo zero sai cedo: só a carteira é gravada.
- Saldo positivo grava a `OPENING`, o lançamento e os dois eventos. Qualquer erro desfaz tudo, inclusive a carteira.
- Jogador e moeda repetidos: o `INSERT` viola o `UNIQUE`, o repositório devolve `ErrWalletAlreadyExists` e o handler responde `409`. Não há consulta prévia "já existe?": duas requisições simultâneas passariam por ela.

> **Por que a versão é 1 e não 2.** A carteira já nasce com o saldo. Se o caso de uso criasse com zero e chamasse `wallet.Credit`, a versão subiria para 2, e o desafio fixa 1 na abertura. Por isso o lançamento de abertura é montado direto com `NewLedgerEntry` (de 0.00 para o saldo inicial).

### Relógio e IDs como dependência

```
type Clock interface       { Now() time.Time }
type IDGenerator interface { NewID() string }
```

O domínio recebe `id` e `now` por parâmetro (spec 03). Quem os fornece é o caso de uso, por duas interfaces pequenas. Em produção são o relógio do sistema e UUID v7; em teste podem ser valores fixos.

O relógio trunca em microssegundo, que é a precisão do PostgreSQL. Sem isso, o horário em memória teria nanossegundos e o relido do banco não seria igual.

## 5. Eventos

```
type EventEnvelope struct {
    EventID       string    `json:"eventId"`
    EventType     string    `json:"eventType"`
    AggregateID   string    `json:"aggregateId"`
    CorrelationID string    `json:"correlationId"`
    CausationID   string    `json:"causationId,omitempty"`
    OccurredAt    time.Time `json:"occurredAt"`
    Version       int       `json:"version"`
}

type WalletBalanceChanged struct {
    EventEnvelope
    Data WalletBalanceChangedData `json:"data"`
}
```

- **Struct tags** (o texto entre crases) dizem ao `encoding/json` o nome de cada campo. É o `[JsonPropertyName]` do C#. `omitempty` omite o campo quando está vazio.
- **Embedding:** `EventEnvelope` aparece sem nome de campo dentro de `WalletBalanceChanged`. Os campos dele sobem para o nível do evento, no código e no JSON. É composição, não herança, mas o efeito no JSON é o de uma classe base.
- Cada evento tem seu tipo e seu construtor. O construtor define `eventType` e `version`; quem chama não escolhe.

| CAMPO | VALOR |
|---|---|
| `aggregateId` | Sempre o `walletId`: mantém a ordem dos eventos de uma carteira na fila FIFO |
| `correlationId` | O ID da requisição que originou tudo |
| `causationId` | O ID da transação que causou o evento |
| `occurredAt` | UTC em RFC 3339 |

O evento é serializado e gravado em `outbox_events` **dentro da transação**. Nada é publicado aqui: a publicação no SQS é da spec 10. É isso que impede evento antes do commit, que é eliminatório.

## 6. Handlers HTTP

```
func (h *WalletHandler) Get(w http.ResponseWriter, r *http.Request) {
    walletID, err := walletIDParam(r)
    if err != nil {
        writeError(w, r, h.logger, err)
        return
    }
    wallet, err := h.get.Execute(r.Context(), walletID)
    if err != nil {
        writeError(w, r, h.logger, err)
        return
    }
    writeJSON(w, http.StatusOK, toWalletResponse(wallet))
}
```

- Todo handler em Go tem esta assinatura: recebe onde escrever a resposta (`w`) e a requisição (`r`). Não há valor de retorno; é preciso dar `return` depois de escrever o erro.
- `r.Context()` é cancelado se o cliente desconectar. Ele desce até a query no banco.
- Os tipos `walletResponse` e `openWalletRequest` são o contrato JSON. O domínio não tem struct tags de API: o handler converte.

### Decodificação estrita

```
decoder := json.NewDecoder(http.MaxBytesReader(w, r.Body, maxRequestBodyBytes))
decoder.DisallowUnknownFields()
```

Corpo acima de 1 MB, campo desconhecido ou dois objetos seguidos resultam em `400`. O valor monetário passa pelo `UnmarshalJSON` do `Money` (spec 02), então `10.5` como número JSON ou `"10.000"` são recusados já na decodificação.

```
InitialBalance *domain.Money `json:"initialBalance"`
```

O ponteiro distingue "campo ausente" (`nil`) de "campo presente".

### Middleware

```
func correlationID(ids application.IDGenerator) func(http.Handler) http.Handler {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            // ... antes ...
            next.ServeHTTP(w, r.WithContext(ctx))
            // ... depois ...
        })
    }
}
```

Um middleware é uma função que recebe o próximo handler e devolve outro que o envolve. É o mesmo conceito do pipeline do ASP.NET Core ou dos middlewares do Express. `router.Use(...)` aplica a todas as rotas.

## 7. Erro vira status em um lugar só

| ERRO | HTTP | CÓDIGO |
|---|---|---|
| `*domain.ValidationError`, erros do `Money` | 400 | `INVALID_REQUEST` |
| `ErrWalletNotFound` | 404 | `WALLET_NOT_FOUND` |
| `ErrWalletAlreadyExists` | 409 | `WALLET_ALREADY_EXISTS` |
| `ErrUnavailable`, prazo expirado | 503 + `Retry-After` | `SERVICE_UNAVAILABLE` |
| Qualquer outro | 500 | `INTERNAL_ERROR` |

O handler só chama `writeError`. A tabela fica em `errors.go`, e é aqui que a separação transitório/permanente da spec 05 chega ao cliente.

> **Erro 500 não vaza detalhe.** A mensagem ao cliente é sempre `internal error`. A causa real, por exemplo um erro de SQL, vai só para o log, junto com o correlation ID.

## 8. Paginação por cursor

```
SELECT ... FROM wallet_ledger_entries
 WHERE wallet_id = $1 AND sequence > $2
 ORDER BY sequence
 LIMIT $3
```

| | OFFSET (`OFFSET 100`) | CURSOR (`SEQUENCE > 100`) |
|---|---|---|
| Linha nova inserida durante a leitura | Pode repetir ou pular itens | Não afeta as páginas já lidas |
| Custo na página 1000 | O banco lê e descarta as anteriores | Vai direto pelo índice |

- O cursor é o `sequence` do último item, em base64url com o prefixo `v1:`. O cliente só o devolve; não interpreta.
- A consulta pede `limit + 1` registros. Se vier o extra, existe próxima página, e ele é descartado.
- Cursor que a API não emitiu resulta em `400`.
- Antes de listar, o caso de uso confere se a carteira existe. Sem isso, carteira inexistente pareceria carteira com ledger vazio.

```
response := ledgerPageResponse{Items: make([]ledgerEntryResponse, len(page.Entries))}
```

Um slice `nil` vira `null` no JSON; um slice vazio criado com `make` vira `[]`. O contrato devolve sempre `[]`.

## 9. Ligação no Fx

```
fx.Provide(
    application.NewOpenWallet,
    application.NewGetWallet,
    application.NewListLedger,
),
```

Os casos de uso são registrados em `internal/app`, não no pacote `application`: assim `application` continua sem importar o Fx. O Fx lê os parâmetros de `NewOpenWallet` (sete interfaces) e entrega as implementações registradas com `fx.As` na spec 05.

## 10. Testes

```
request := httptest.NewRequest(http.MethodPost, "/wallets", strings.NewReader(body))
recorder := httptest.NewRecorder()
env.router.ServeHTTP(recorder, request)
// recorder.Code, recorder.Body, recorder.Header()
```

`httptest` chama o router direto, sem abrir porta. É o `WebApplicationFactory` do ASP.NET ou o `supertest` do Node. O router, os casos de uso e o banco são os reais.

### Como provar "tudo ou nada"

```
type failingOutbox struct {
    application.OutboxRepository
    calls int
}

func (o *failingOutbox) Append(ctx context.Context, m application.OutboxMessage) error {
    o.calls++
    if o.calls == 2 {
        return errOutboxDown
    }
    return o.OutboxRepository.Append(ctx, m)
}
```

A struct embute a interface e sobrescreve um método. Ela falha no segundo evento, quando carteira, transação, lançamento e o primeiro evento já foram gravados. O teste confere que as quatro tabelas terminam vazias.

`decode[T any]` no teste é uma função genérica: `decode[walletResponse](t, response)`. Mesma ideia de `Deserialize<T>`.

## 11. Como foi verificado

| VERIFICAÇÃO | RESULTADO |
|---|---|
| Abrir com 1000.00 | 201; 1 carteira, 1 `OPENING`, 1 crédito de 0.00 para 1000.00, 2 eventos com o correlation ID enviado |
| Abrir com 0.00 | Só a carteira; ledger responde `{"items":[],"nextCursor":null}` |
| Mesma abertura de novo | 409; nada novo gravado. Mesma pessoa em outra moeda: 201 |
| 12 corpos inválidos | 400 em todos; nenhuma carteira criada |
| Outbox falhando no segundo evento | Quatro tabelas vazias |
| 9 lançamentos, páginas de 3, inserção no meio | Ordem lida igual à do banco, sem repetir nem pular |
| `curl` no Docker Compose | Mesmos resultados; eventos conferidos na tabela `outbox_events` |
| `go test -race` (container); cobertura de `internal/` | Passou; 90,6% |

> **Ainda sem autenticação.** Estes endpoints estão abertos até a spec 09, que restringe `/wallets` ao serviço interno.

## 12. Perguntas de revisão

**Por que não consultar "já existe carteira?" antes de inserir?**

Duas requisições simultâneas passariam pela consulta. O `UNIQUE` do banco é a única checagem que vale entre instâncias.

**O que garante que carteira, transação, lançamento e eventos são gravados juntos?**

Tudo acontece dentro de um `WithinTx`. Qualquer erro retornado faz rollback de todos.

**Por que o evento vai para uma tabela em vez de ir direto para o SQS?**

Para ser confirmado no mesmo commit do saldo. Publicar direto permitiria evento sem commit, ou commit sem evento.

**O que é embedding e o que ele causa no JSON?**

Colocar um tipo dentro de uma struct sem nome de campo. Os campos sobem de nível: o envelope fica ao lado de `data`.

**Qual a vantagem do cursor sobre o offset?**

Inserções durante a leitura não fazem repetir nem pular itens, e páginas distantes custam o mesmo que a primeira.

**Por que a consulta do ledger pede `limit + 1`?**

O registro extra indica se há próxima página sem uma segunda consulta.

### Por que `Clock` é uma interface?

Para o caso de uso não ler o relógio direto. Em teste dá para fixar o horário; em produção ele é truncado na precisão do banco.

### Onde um erro vira status HTTP?

Em `toAPIError`, em `errors.go`. Os handlers só chamam `writeError`.

## Ordem de leitura do código

1. `internal/infra/httpapi/server.go`: as rotas
2. `internal/infra/httpapi/wallets.go`
3. `internal/application/open_wallet.go`
4. `internal/domain/events.go`
5. `internal/application/list_ledger.go`
6. `internal/infra/httpapi/errors.go` e `middleware.go`
7. `internal/infra/httpapi/wallets_test.go`

SPEC 07

# Operações e idempotência

O coração do desafio: um único caso de uso que aplica cada operação exatamente uma vez, mesmo que ela chegue 50 vezes ao mesmo tempo, em instâncias diferentes, ou de novo depois de um reinício.

## 1. O que a spec pedia

- `POST /wagering/transactions` com o header `Idempotency-Key` obrigatório, e duas consultas de transação.
- Idempotência gravada no banco, com hash determinístico dos campos de negócio.
- Mesma chave e mesmo conteúdo: devolve o resultado original. Mesma chave e conteúdo diferente: conflito.
- A mesma operação `(providerId, externalTransactionId)` não pode ser reaplicada com outra chave.
- Replay devolve o saldo observado no processamento original.
- Duas apostas de 80 sobre 100, ao mesmo tempo: uma processada, uma rejeitada, saldo 20, um débito.

## 2. O que foi construído

| ARQUIVO | CONTEÚDO |
|---|---|
| `application/process_wager_transaction.go` | O caso de uso `ProcessWagerTransaction` |
| `application/payload_hash.go` | O hash do payload |
| `application/get_wager_transaction.go` | Consulta por ID interno e por ID externo |
| `infra/httpapi/wagering.go` | Os três handlers e o contrato JSON |

O domínio não mudou: `ApplyWager` da spec 03 já decidia tudo. Esta spec cuida do que fica em volta: achar duplicata, travar, gravar.

## 3. O que é idempotência

Uma operação é idempotente quando executá-la duas vezes tem o mesmo efeito de executá-la uma. Redes falham: o provedor manda uma aposta, não recebe a resposta e manda de novo. O saldo só pode ser debitado uma vez, e as duas respostas precisam ser iguais.

Para isso cada requisição traz uma **chave de idempotência**, e o servidor guarda, junto com a transação, um **hash do conteúdo**.

| CHEGOU | RESPOSTA |
|---|---|
| Chave nova, operação nova | Processa |
| Mesma chave, mesmo hash | Resultado gravado, `idempotentReplay: true` |
| Mesma chave, hash diferente | `409 IDEMPOTENCY_KEY_CONFLICT` |
| Chave nova, mas `externalTransactionId` já usado | `409 EXTERNAL_TRANSACTION_CONFLICT` |

O hash existe por causa da terceira linha: sem ele, um cliente com bug reutilizando a chave receberia como resposta o resultado de outra operação, sem perceber.

## 4. O hash do payload

```
func payloadHash(in ProcessWagerInput) (string, error) {
    fields := map[string]any{
        "providerId":            in.ProviderID,
        "externalTransactionId": in.ExternalTransactionID,
        "playerId":              in.PlayerID,
        "walletId":              in.WalletID,
        "roundId":               in.RoundID,
        "gameId":                in.GameID,
        "kind":                  in.Kind,
        "money": map[string]string{"amount": in.Money.Amount(), "currency":
in.Money.Currency()},
    }
    if in.ReferenceExternalTransactionID != "" {
        fields["referenceExternalTransactionId"] = in.ReferenceExternalTransactionID
    }

    canonical, err := json.Marshal(fields)
    ...
    sum := sha256.Sum256(canonical)
    return hex.EncodeToString(sum[:]), nil
}
```

- **JSON canônico:** a mesma informação precisa virar sempre os mesmos bytes. `{"a":1,"b":2}` e `{"b":2,"a":1}` são o mesmo objeto, mas dariam hashes diferentes. O `encoding/json` escreve as chaves de um `map` em ordem alfabética, e é isso que torna a saída canônica.
- **Normalização:** `in.Money.Amount()` devolve sempre duas casas. `"25"`, `"25.0"` e `"25.00"` geram o mesmo hash.
- **Fora do hash:** a chave de idempotência e o correlation ID. São do transporte, não da operação. Por isso HTTP e SQS geram o mesmo hash para o mesmo conteúdo.
- `sha256.Sum256` devolve um array de 32 bytes; `sum[:]` o transforma em slice para o `hex`.

Um teste fixa o hash da aposta do enunciado em um valor calculado por fora (`sha256sum`). Se alguém mudar o algoritmo, o teste falha: mudar o hash invalidaria todos os registros já gravados.

## 5. O caso de uso, passo a passo

1. Valida a entrada e calcula o hash.
2. Abre a transação SQL.
3. `GetForUpdate` na carteira: trava a linha.
4. Procura duplicata: pela chave, depois pelo ID externo.
5. Resolve a referência, se houver.
6. `domain.ApplyWager` decide: processa, rejeita ou espera.
7. Grava a transação. Se o saldo mudou, grava saldo e lançamento.
8. Grava os eventos na outbox.
9. Commit.

Tudo entre os passos 2 e 9 é uma transação só. Uma rejeição de negócio também percorre o caminho inteiro: é gravada como `REJECTED`, gera evento e pode ser devolvida igual em um replay.

### Por que o lock vem antes da checagem de duplicata

| INSTANTE | REQUISIÇÃO A | REQUISIÇÃO B (IDÊNTICA) |
|---|---|---|
| 1 | Trava a carteira | |
| 2 | Duplicata? Não | Tenta travar: **espera** |
| 3 | Debita, grava, commit | |
| 4 | | Trava a carteira |
| 5 | | Duplicata? **Sim**: devolve o resultado de A |

Com a ordem invertida, as duas perguntariam "já existe?" antes de qualquer uma gravar, as duas ouviriam "não", e as duas tentariam debitar.

### Quando o lock não basta

O lock é por carteira. Se a mesma chave chegar com `walletId` diferentes, cada requisição trava uma carteira e nenhuma espera a outra. Quem decide é o `UNIQUE (provider_id, idempotency_key)` da spec 04:

```
for attempt := 1; ; attempt++ {
    err = uc.tx.WithinTx(ctx, func(ctx context.Context) error { ... })
    if !lostDuplicateRace(err) || attempt == maxProcessAttempts {
        break
    }
}
```

1. O `INSERT` perdedor viola o `UNIQUE`.
2. O repositório devolve `ErrDuplicateIdempotencyKey` e a transação inteira sofre rollback.
3. O caso de uso tenta de novo, em uma transação nova.
4. Agora a checagem de duplicata encontra a linha: replay ou conflito.

É a regra da spec 05 em uso: depois de uma violação de constraint, a transação está abortada, então não se continua nela.

> **Duas camadas.** O lock resolve o caso comum sem erro nenhum. A constraint resolve o caso que o lock não cobre. Nenhuma das duas depende de memória do processo, então funciona igual com três instâncias.

## 6. O saldo do replay

A coluna `result_balance_minor` guarda o saldo que a operação observou. O replay lê esse valor, não o saldo atual da carteira.

| PASSO | SALDO DA CARTEIRA | RESPOSTA |
|---|---|---|
| Aposta de 25.00 | 975.00 | `balance: 975.00` |
| Outra aposta de 100.00 | 875.00 | |
| Ganho de 500.00 | 1375.00 | |
| Replay da primeira aposta | 1375.00 | `balance: 975.00`, `idempotentReplay: true` |

## 7. Contrato HTTP

| SITUAÇÃO | HTTP | CORPO |
|---|---|---|
| Processada | 200 | `status: PROCESSED`, `balance` |
| Rejeição de negócio | 422 | `status: REJECTED`, `failureCode`, `balance` |
| Aguardando referência | 202 | `status: PENDING_REFERENCE` |
| Conflito de chave ou de operação | 409 | `error.code` |
| Entrada inválida | 400 | `error.code`, `field` |
| Carteira inexistente | 404 | `error.code` |
| Banco indisponível | 503 + `Retry-After` | `error.code` |

> **400 × 422.** `400` é entrada corrigível: nada é gravado e o provedor pode reenviar com a mesma chave depois de corrigir. `422` é resultado definitivo: a transação foi gravada como `REJECTED`, e reenviar devolve a mesma rejeição.

```
func statusFor(status domain.TransactionStatus) int {
    switch status {
    case domain.StatusProcessed:
        return http.StatusOK
    case domain.StatusRejected, domain.StatusFailed:
        return http.StatusUnprocessableEntity
    default:
        return http.StatusAccepted
    }
}
```

O replay usa a mesma função, então devolve o mesmo código HTTP do original.

```
Balance *domain.Money `json:"balance,omitempty"`
```

Ponteiro com `omitempty`: quando não há saldo observado (operação aguardando referência), o campo não aparece no JSON.

## 8. Testes de concorrência

```
results := make(chan *httptest.ResponseRecorder, requests)
var wg sync.WaitGroup
for range requests {
    wg.Go(func() { results <- env.send("provider-a:transaction-123", bet) })
}
wg.Wait()
close(results)
```

- Cinquenta goroutines enviam a mesma aposta. Cada uma escreve a resposta em um channel com espaço para 50 (*buffered*), então nenhuma fica bloqueada ao enviar.
- Depois de `close`, o `for response := range results` lê o que ficou no channel e termina.
- O teste exige: 50 respostas `200`, um único `transactionId`, exatamente uma com `idempotentReplay: false` e um débito no ledger.

### Simulando outra instância

A corrida entre carteiras diferentes depende de milissegundos: não dá para reproduzir só disparando goroutines. O teste a força com um repositório falso que, no primeiro `Insert`, grava antes uma transação "gêmea" por outra conexão:

```
func (r *racingWagers) Insert(ctx context.Context, tx *domain.WagerTransaction) error {
    if !r.raced {
        r.raced = true
        // context.Background() não carrega transação: a gêmea é confirmada em outra conexão
        postgres.NewWagerTransactionRepository(r.pool).Insert(context.Background(), r.twin(tx))
    }
    return r.WagerTransactionRepository.Insert(ctx, tx)
}
```

O `Insert` verdadeiro então viola o `UNIQUE`. O teste confere que a tentativa perdedora foi desfeita por inteiro (saldo intacto, nenhum lançamento, nenhum evento) e que o resultado final é replay ou conflito.

Ao escrever esse teste apareceu um detalhe do PostgreSQL: a gêmea na **mesma** carteira ficaria travada para sempre. Inserir uma linha com chave estrangeira pede um lock leve na linha referenciada, e o `FOR UPDATE` bloqueia até esse. Ou seja, com o lock da carteira preso, nenhuma outra transação consegue nem inserir uma operação para ela.

## 9. Como foi verificado

| VERIFICAÇÃO | RESULTADO |
|---|---|
| Mesma aposta 50 vezes em paralelo | Um débito; saldo 975.00, versão 2 |
| Duas apostas de 80.00 sobre 100.00 em paralelo | Uma `PROCESSED`, uma `REJECTED` / `INSUFFICIENT_FUNDS`; saldo 20.00; um débito |
| Reenvio das duas | Mesmos resultados, mesmo saldo, nada novo gravado |
| Replay após outras movimentações | Devolve 975.00 com a carteira em 1375.00 |
| Mesma chave com 6 conteúdos diferentes | `409` em todos |
| 5 carteiras, 10 apostas simultâneas cada | Todas com 90.00 |
| 19 entradas inválidas | `400` ou `404`; nenhuma transação gravada |
| `curl` no Compose, com reinício da aplicação | Replay continua devolvendo o resultado original |
| `go test -race`, concorrência repetida 5 vezes | Passou |

Os testes usam várias conexões dentro de um processo. A prova com três processos independentes é da spec 15.

> **Ainda sem autenticação.** Hoje qualquer chamador informa qualquer `providerId`. A spec 09 amarra o `providerId` ao token.

## 10. Perguntas de revisão

**Para que serve o hash, se já existe a chave de idempotência?**

Para detectar a mesma chave com conteúdo diferente. Sem ele, o servidor devolveria o resultado de outra operação.

**O que torna o JSON canônico?**

Chaves em ordem fixa (alfabética) e valor monetário normalizado. A mesma operação gera sempre os mesmos bytes.

**Por que a chave de idempotência não entra no hash?**

Ela é do transporte. O hash descreve a operação, e precisa ser igual por HTTP e por SQS.

**Por que travar a carteira antes de procurar duplicata?**

Para que a segunda requisição idêntica só procure depois que a primeira gravou.

**O que acontece quando a mesma chave chega para carteiras diferentes ao mesmo tempo?**

O lock não serializa. O `UNIQUE` recusa o segundo `INSERT`, a transação é desfeita e a nova tentativa encontra a linha.

**De onde vem o saldo devolvido em um replay?**

Da coluna `result_balance_minor` da transação, gravada no processamento original.

**Aposta sem saldo responde 400 ou 422? Por quê?**

`422`. É um resultado de negócio definitivo, gravado como `REJECTED`. `400` é entrada malformada, que não é gravada.

**A idempotência sobrevive a um reinício?**

Sim. Chave, hash e resultado estão no PostgreSQL; nada fica em memória.

### Ordem de leitura do código

1. `internal/application/payload_hash.go` e seu teste
2. `internal/application/process_wager_transaction.go`: `Execute`, depois `process`, depois `findDuplicate`
3. `internal/infra/httpapi/wagering.go`
4. `internal/infra/httpapi/wagering_test.go`: os dois testes com `InParallel` no nome
5. `internal/application/process_wager_transaction_test.go`

SPEC 08

# Reversões e referências pendentes

Um `REFUND` pode chegar antes da aposta que ele devolve. Em vez de recusar, o serviço guarda a operação como pendente e um worker em segundo plano tenta de novo, com intervalos cada vez maiores, até a aposta chegar ou o prazo acabar.

## 1. O que a spec pedia

- Referência não encontrada: gravar `PENDING_REFERENCE` e responder `202` (já feito na spec 07).
- Um worker que tenta de novo com **backoff exponencial**.
- O estado da espera fica no banco e sobrevive a reinício.
- Várias instâncias do worker não processam a mesma pendência duas vezes.
- Esgotado o limite: `REJECTED` com `REFERENCE_NOT_FOUND`.
- Dois `REFUND` da mesma aposta em paralelo: um processado, um rejeitado.

## 2. O que foi construído

| ARQUIVO | CONTEÚDO |
|---|---|
| `migrations/000005_add_reference_retry` | Três colunas, dois `CHECK`, dois índices parciais |
| `domain/wager_transaction.go` | `ScheduleReferenceAttempt` e os campos da espera |
| `application/resolve_pending_references.go` | `ReferenceRetryPolicy` e o caso de uso `ResolvePendingReferences` |
| `infra/postgres/wager_transaction_repository.go` | `ClaimPendingReferences` e `WakePendingReferences` |
| `infra/worker/` | A goroutine que chama o caso de uso a cada intervalo (na spec 10 virou o `Poller` genérico de `poller.go`) |

As regras de reversão (quem pode reverter o quê, valor igual, uma reversão por transação) continuam em `ApplyWager`, da spec 03. O worker chama a mesma função que o endpoint HTTP chama.

## 3. Onde fica o estado da espera

Se a contagem de tentativas ficasse em memória, um reinício zeraria tudo. Por isso ela fica em três colunas novas de `wager_transactions`:

| COLUNA | USO |
|---|---|
| `reference_attempts` | Quantas consultas não acharam a referência |
| `next_reference_attempt_at` | Quando consultar de novo |
| `correlation_id` | O correlation ID da requisição original |

```
CONSTRAINT wager_transactions_next_attempt_only_while_waiting CHECK (
    (status = 'PENDING_REFERENCE') = (next_reference_attempt_at IS NOT NULL)
)
```

O `CHECK` compara dois booleanos: a data existe **se e somente se** a transação está esperando. Uma pendência sem data nunca seria pega de novo pelo worker; o banco recusa esse estado.

O `correlation_id` existe porque o worker conclui a operação minutos depois, fora de qualquer requisição. Sem a coluna, os eventos emitidos por ele não poderiam ser ligados ao pedido original.

## 4. Backoff exponencial

Consultar o banco a cada segundo por uma referência que pode demorar minutos é desperdício. No backoff exponencial a espera dobra a cada falha, até um teto.

```
func (p ReferenceRetryPolicy) delayAfter(attempts int) time.Duration {
    delay := p.BaseDelay
    for i := 1; i < attempts; i++ {
        if delay >= p.MaxDelay/2 {
            return p.MaxDelay
        }
        delay *= 2
    }
    return min(delay, p.MaxDelay)
}
```

| TENTATIVA QUE FALHOU | 1 | 2 | 3 | 4 | 5 | 6 | 7 A 9 |
|---|---|---|---|---|---|---|---|
| Espera até a próxima | 1s | 2s | 4s | 8s | 16s | 32s | 60s |

- Com os padrões (base 1s, teto 1min, 10 tentativas) uma referência que nunca chega expira em cerca de 4 minutos.
- O `if` dentro do laço testa **antes** de dobrar. `time.Duration` é um `int64` de nanossegundos; dobrar sem testar estouraria com um número alto de tentativas.
- `min` é função embutida do Go desde a versão 1.21.
- Os quatro valores vêm de variáveis de ambiente ( `REFERENCE_*` ).

## 5. Uma fila dentro do PostgreSQL

As pendências formam uma fila: "tudo que está `PENDING_REFERENCE` e com a data vencida". Com três instâncias do serviço, três workers leem essa fila ao mesmo tempo. O problema é entregar cada item a um só.

```
UPDATE wager_transactions
   SET next_reference_attempt_at = $2          -- agora + 30s
 WHERE id IN (
    SELECT id FROM wager_transactions
     WHERE status = 'PENDING_REFERENCE' AND next_reference_attempt_at <= $1
     ORDER BY next_reference_attempt_at
     LIMIT $3
       FOR UPDATE SKIP LOCKED
 )
RETURNING id::text, wallet_id::text
```

- `FOR UPDATE` trava as linhas escolhidas.
- `SKIP LOCKED` faz o comando **pular** as linhas que outra transação já travou, em vez de esperar por elas. Dois workers rodando ao mesmo tempo recebem conjuntos diferentes.
- `RETURNING` devolve as linhas alteradas, como um `SELECT` embutido no `UPDATE`.

O lock some quando esse comando termina. O que protege a pendência depois disso é a data empurrada 30 segundos para a frente: para os outros workers ela deixou de estar vencida. Isso é uma **reserva com prazo** (em inglês, *lease*).

> **Por que o prazo.** Se a instância morrer logo depois de reservar, ninguém precisa "liberar" nada: em 30 segundos a data vence e outra instância pega a pendência. Um teste simula isso reservando sem resolver.

## 6. Resolver uma pendência

1. Abrir transação SQL e travar a carteira ( `GetForUpdate` ).
2. Reler a transação. Se não está mais `PENDING_REFERENCE`, outra instância já resolveu: sair.
3. Procurar a referência e chamar `domain.ApplyWager`.
4. Ainda esperando e há tentativas: somar uma tentativa, gravar a próxima data, sair.
5. Ainda esperando e sem tentativas: marcar `REJECTED` com `REFERENCE_NOT_FOUND`.
6. Gravar transação, saldo, lançamento e eventos.

Os passos de 3 em diante usam o mesmo código do endpoint HTTP. A parte comum foi extraída para `wagerOutcome.record`, chamada pelos dois casos de uso.

> **Ordem dos locks.** O worker trava a carteira e só depois mexe na linha da transação, na mesma ordem do endpoint HTTP. Se ele segurasse a linha da transação e depois pedisse a carteira, poderia cruzar com uma requisição que tem a carteira e quer alterar aquela linha: cada um esperando o outro, um *deadlock*. Por isso a reserva (passo anterior) solta o lock da linha antes de a resolução começar.

## 7. Despertar: não esperar o backoff à toa

O backoff serve para a referência que não chegou. Quando ela chega, esperar o resto do intervalo seria atraso sem motivo. Ao gravar uma transação em estado final, a mesma transação SQL antecipa as pendências que apontam para ela:

```
UPDATE wager_transactions
   SET next_reference_attempt_at = $3          -- agora
 WHERE provider_id = $1 AND reference_external_transaction_id = $2
   AND status = 'PENDING_REFERENCE' AND next_reference_attempt_at > $3
```

O worker pega a pendência no ciclo seguinte (até 1 segundo). Isso também resolve cadeias: um `ROLLBACK` esperando um `REFUND` que espera uma `BET` é concluído em dois ciclos.

## 8. O worker: goroutine, ticker e cancelamento

```
func (w *ReferenceResolver) Run(ctx context.Context) {
    ticker := time.NewTicker(w.interval)
    defer ticker.Stop()

    for {
        w.drain(ctx)
        select {
        case <-ctx.Done():
            return
        case <-ticker.C:
        }
    }
}
```

- `time.NewTicker` cria um canal ( `ticker.C` ) que recebe um valor a cada intervalo. É o equivalente a um `setInterval` ou a um `PeriodicTimer` do C#.
- `select` espera o que acontecer primeiro: o contexto ser cancelado ou o próximo tique.
- `drain` repete enquanto o lote volta cheio, e para depois de um erro, para não girar em falso com o banco fora do ar.

```
ctx, cancel := context.WithCancel(context.Background())
done := make(chan struct{})

lc.Append(fx.Hook{
    OnStart: func(context.Context) error {
        go func() {
            defer close(done)
            w.Run(ctx)
        }()
        return nil
    },
    OnStop: func(stopCtx context.Context) error {
        cancel()
        select {
        case <-done:
            return nil
        case <-stopCtx.Done():
            return stopCtx.Err()
        }
    },
})
```

- O contexto que o Fx passa ao `OnStart` expira quando a inicialização termina. A goroutine precisa de um contexto próprio.
- `done` é um canal fechado quando `Run` retorna. No `OnStop`, `cancel()` pede a parada e `<-done` espera a pendência em andamento terminar.
- Se demorar mais que o prazo de desligamento, `stopCtx.Done()` vence e o Fx segue. A transação SQL interrompida é desfeita e a reserva vence sozinha.

## 9. Como foi testado

| CENÁRIO | RESULTADO VERIFICADO |
|---|---|
| `REFUND` antes da `BET` | `202`; a aposta chega; o resolvedor conclui; saldo volta a 100.00; eventos com o correlation ID original |
| Referência que nunca chega | Esperas de 1s, 2s, 4s, 4s; não é pega antes da hora; termina `REJECTED` / `REFERENCE_NOT_FOUND` |
| Referência rejeitada | A reversão vira `REFERENCE_NOT_PROCESSED` |
| `ROLLBACK` de `WIN` sem saldo | `REVERSAL_INSUFFICIENT_FUNDS` |
| Dois `REFUND` (ou `REFUND` + `ROLLBACK`) em paralelo | Um `PROCESSED`, um `ALREADY_REVERSED`, um crédito |
| 4 instâncias, 40 pendências, lotes de 5 | Soma das reservas = 40; 40 créditos no razão |
| Instância morre depois de reservar | Invisível durante o prazo; retomada depois dele |

- Os testes usam um relógio com deslocamento ( `testClock.advance` ) para atravessar o backoff sem `sleep` .
- Removi de propósito a reserva e depois o despertar: os testes falharam nos dois casos, e voltei o código.
- `go test -race` passa, com os testes de concorrência repetidos 5 vezes.
- Com `curl` no Docker Compose: pendência criada, aplicação reiniciada, aposta enviada, `REFUND` concluído; outra pendência expirou com `REFERENCE_NOT_FOUND` .

> **Limite.** A disputa foi testada com vários resolvedores no mesmo processo, cada um com sua conexão. A prova com três processos separados fica para a spec 15.

## 10. Para fixar

**O que `SKIP LOCKED` faz de diferente de um `FOR UPDATE` comum?**

Pula as linhas já travadas em vez de esperar. Cada worker recebe linhas diferentes e nenhum fica parado.

**O lock do `SKIP LOCKED` dura um comando. O que impede outro worker de pegar a pendência logo depois?**

A data da próxima tentativa foi empurrada 30 segundos. Para os outros, ela não está vencida.

**O que acontece se a instância morrer com uma pendência reservada?**

Nada precisa ser liberado. A reserva vence e outra instância assume.

**Por que o worker trava a carteira antes de mexer na transação?**

Para usar a mesma ordem de locks do endpoint HTTP e não haver deadlock; e para que reversões da mesma carteira sejam decididas uma de cada vez.

**Se a aposta chega, o `REFUND` espera o backoff terminar?**

Não. A gravação da aposta antecipa a data das pendências que a referenciam.

**Por que guardar o `correlation_id` na transação?**

O worker roda fora de qualquer requisição; sem a coluna, os eventos dele não se ligariam ao pedido original.

### Ordem de leitura do código

1. `migrations/000005_add_reference_retry.up.sql`
2. `internal/application/resolve_pending_references.go` : a política, depois `Execute` e `resolve`
3. `internal/infra/postgres/wager_transaction_repository.go` : `ClaimPendingReferences` e `WakePendingReferences`
4. `internal/infra/worker/poller.go` e `module.go`
5. `internal/infra/httpapi/references_test.go`

SPEC 09

# Autenticação e autorização

Até aqui qualquer um podia chamar qualquer endpoint dizendo ser qualquer provedor. Agora todo endpoint de negócio exige um token emitido pelo Keycloak, e é o token, não o corpo da requisição, que diz quem está chamando.

## 1. O que a spec pedia

- Provedor de identidade externo (OAuth 2.0 / OIDC), fluxo `client_credentials`. Sem senha e sem token próprio.
- O `providerId` do corpo precisa bater com o do token.
- Provedor só vê as próprias transações; carteira só para o serviço interno; health checks públicos.
- Sem token ou token inválido: `401`. Token válido sem permissão: `403`. Transação de outro provedor: `404`.

> **Eliminatório.** Endpoint de negócio sem autenticação efetiva, ou acesso a transação de outro provedor, desclassifica a entrega.

## 2. Autenticação × autorização

| | PERGUNTA | FALHA |
|---|---|---|
| **Autenticação** | Quem é você? O token é legítimo? | `401 Unauthorized` |
| **Autorização** | Você pode fazer isto? | `403 Forbidden` |

O nome do `401` engana: ele significa "não autenticado". O `403` é o "não autorizado" de verdade.

## 3. O fluxo `client_credentials`

É o fluxo OAuth para serviço falando com serviço, sem usuário humano. Cada provedor é um **client** no Keycloak, com um identificador e um segredo.

1. O provedor manda `client_id` e `client_secret` ao Keycloak.
2. O Keycloak devolve um **access token** (um JWT) válido por 5 minutos.
3. O provedor chama o nosso serviço com `Authorization: Bearer <token>`.
4. O serviço confere o token sozinho, sem chamar o Keycloak a cada requisição.

```
curl -s -X POST http://localhost:8081/realms/wagering/protocol/openid-connect/token \
  -d grant_type=client_credentials -d client_id=provider-a -d client_secret=provider-a-secret
```

## 4. JWT e JWKS

Um JWT tem três partes em base64url separadas por ponto: `cabeçalho.conteúdo.assinatura`. O conteúdo de um token do `provider-a`:

```
{
  "iss": "http://localhost:8081/realms/wagering",
  "aud": "wagering-service",
  "azp": "provider-a",
  "exp": 1791979240,
  "realm_access": { "roles": ["provider"] },
  "provider_id": "provider-a"
}
```

- `iss` (issuer): quem emitiu. `aud` (audience): para quem o token foi feito. `exp`: quando expira.
- `azp`: o client que pediu o token. `realm_access.roles`: os papéis. `provider_id`: claim que criamos no Keycloak.
- Qualquer um lê o conteúdo; ele não é secreto. O que impede a falsificação é a **assinatura**.

O Keycloak assina com uma chave privada RSA que só ele tem. A chave pública correspondente fica publicada em um endereço chamado **JWKS** (JSON Web Key Set). O nosso serviço baixa essas chaves e confere a assinatura localmente. O cabeçalho do token traz um `kid` (key id) que diz qual chave usar.

> **Por que conferir `aud`.** O mesmo Keycloak pode emitir tokens para vários sistemas. Sem checar a audiência, um token feito para outro serviço do mesmo realm seria aceito aqui.

## 5. O realm no Keycloak

O arquivo `deploy/keycloak/wagering-realm.json` é importado no `docker compose up`. Ele cria:

| CLIENT | PAPEL | MAPPERS |
|---|---|---|
| `provider-a`, `provider-b` | `provider` | audiência `wagering-service`; claim fixo `provider_id` |
| `wallet-service` | `internal` | audiência `wagering-service` |

- Um **mapper** é uma regra do Keycloak que acrescenta algo ao token. O de audiência coloca `wagering-service` em `aud`; o *hardcoded claim* coloca `provider_id`.
- Em `client_credentials` o Keycloak cria um usuário técnico por client (`service-account-provider-a`). Os papéis são dados a esse usuário.

### Um detalhe do Docker: dois endereços para o mesmo Keycloak

Do seu computador o Keycloak é `localhost:8081`; de dentro da rede do Docker, a aplicação o encontra como `keycloak:8080`. O `iss` do token depende do endereço usado para pedi-lo, e isso quebraria a validação.

- `KC_HOSTNAME=http://localhost:8081` fixa o `iss` de todos os tokens.
- A aplicação recebe duas configurações: `AUTH_ISSUER` (o `iss` esperado) e `AUTH_JWKS_URL` (de onde baixar as chaves, pela rede interna).

## 6. O verificador

```
keys := oidc.NewRemoteKeySet(context.Background(), cfg.JWKSURL)
verifier := oidc.NewVerifier(cfg.Issuer, keys, &oidc.Config{
    ClientID:             cfg.Audience,
    SupportedSigningAlgs: []string{oidc.RS256},
})
```

- A biblioteca `github.com/coreos/go-oidc/v3` confere assinatura, `iss`, `aud` e `exp`. Não vale escrever isso à mão: erros de validação de token são falhas de segurança.
- `NewRemoteKeySet` baixa as chaves na primeira requisição, guarda em cache e baixa de novo quando aparece um `kid` desconhecido. Isso cobre a **rotação de chaves** do Keycloak.
- Fixar `RS256` impede o ataque clássico do token com `"alg": "none"` (sem assinatura).
- O `context.Background()` é proposital: o conjunto de chaves usa esse contexto nas buscas futuras, e um contexto de requisição seria cancelado logo.

```
type accessTokenClaims struct {
    AuthorizedParty string `json:"azp"`
    ProviderID      string `json:"provider_id"`
    RealmAccess     struct {
        Roles []string `json:"roles"`
    } `json:"realm_access"`
}
```

Depois de validado, o conteúdo vira um `Principal{ClientID, ProviderID, Roles}`. Um token com papel `provider` e sem `provider_id` é recusado: não daria para saber de quais transações ele é dono.

## 7. Middleware em Go e a identidade no `context`

Em Go um middleware é uma função que recebe um `http.Handler` e devolve outro, que faz algo antes (ou em vez) de chamar o próximo. É o mesmo papel do middleware do ASP.NET ou do Express.

```
func authenticate(verifier TokenVerifier, logger *slog.Logger) func(http.Handler) http.Handler {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            raw, ok := bearerToken(r)
            if !ok { /* 401 */ return }
            principal, err := verifier.Verify(r.Context(), raw)
            if err != nil { /* 401 */ return }
            next.ServeHTTP(w, r.WithContext(auth.WithPrincipal(r.Context(), principal)))
        })
    }
}
```

- `r.WithContext` cria uma cópia da requisição com um contexto novo, que carrega o `Principal`. Os handlers o leem com `auth.PrincipalFrom(ctx)`.
- A chave do contexto é um tipo vazio privado (`type principalKey struct{}`). Nenhum outro pacote consegue criar a mesma chave e sobrescrever o valor.

- O middleware depende da interface `TokenVerifier`, não do verificador concreto. Os testes HTTP usam um verificador falso; o verdadeiro é testado no pacote `auth`.

**Rotas com o chi**

```
router.Get("/health/live", health.Live)        // público

router.Group(func(r chi.Router) {
    r.Use(authenticate(verifier, logger))

    r.Group(func(r chi.Router) {
        r.Use(requireRole(logger, auth.RoleInternal))
        r.Post("/wallets", wallets.Open)
        ...
    })
    r.With(requireRole(logger, auth.RoleProvider)).Post("/wagering/transactions",
wagering.Process)
    ...
})
```

`Group` cria um conjunto de rotas que compartilha middlewares; `With` aplica um middleware a uma rota só. Uma rota nova dentro do grupo já nasce protegida.

## 8. As regras de acesso

| SITUAÇÃO | RESPOSTA |
|---|---|
| Sem token, malformado, assinatura inválida, expirado, `iss` ou `aud` errados | `401` + `WWW-Authenticate: Bearer` |
| `provider-a` em `/wallets`; `wallet-service` enviando aposta | `403` |
| `provider-a` enviando `"providerId": "provider-b"` | `403`, nada gravado |
| `provider-b` lendo transação do `provider-a` | `404`, corpo idêntico ao de transação inexistente |

> **Por que 404 e não 403 na consulta.** Um `403` confirmaria que a transação existe. Respondendo exatamente como uma transação inexistente, o provedor não aprende nada sobre os outros. O teste compara os dois corpos byte a byte.

O replay entre provedores ficou resolvido sem código novo: a chave de idempotência é única por provedor, e o provedor agora vem do token. Se `provider-b` usa a mesma chave de `provider-a`, ele cria uma operação dele.

## 9. Como foi testado

- **Verificador:** tokens assinados no próprio teste com uma chave RSA gerada na hora e um servidor JWKS falso (`httptest`). Válido, expirado, outro `iss`, outra `aud`, chave desconhecida, assinatura

adulterada, `alg: none`, sem `provider_id`, texto qualquer. Rotação: o servidor troca a chave e o token novo é aceito; dois tokens com a mesma chave buscam o JWKS uma vez só.

- **HTTP:** todas as seis rotas de negócio sem token e com token falso dão `401`; health dá `200`; os `403` não gravam nada; os `404` têm o mesmo corpo de transação inexistente.
- **Keycloak real:** com `curl` no Docker Compose, os três clients obtêm token; segredo errado não obtém; todos os casos acima repetidos com tokens de verdade.
- `go test -race` passa; cobertura de `internal/` em 89,4%.

> **Limites.** O token expirado só foi testado com tokens assinados no teste. Se o Keycloak estiver fora e chegar um token com chave ainda não vista, a resposta é `401`, não `503`. A política de acesso às filas foi para a spec 11, junto com o LocalStack.

## 10. Para fixar

**Qual a diferença entre 401 e 403?**

`401`: não sei quem você é (sem token ou token inválido). `403`: sei quem você é e você não pode fazer isso.

**Como o serviço confere um token sem chamar o Keycloak?**

Confere a assinatura com a chave pública do JWKS, que fica em cache, e confere `iss`, `aud` e `exp`.

**O que acontece quando o Keycloak troca de chave?**

Chega um token com `kid` desconhecido e o verificador baixa o JWKS de novo.

**Por que o `providerId` do corpo não basta?**

Qualquer um escreve qualquer coisa no corpo. Só o token é assinado.

**Por que transação de outro provedor responde 404?**

Para não revelar que ela existe.

**Por que a chave do `context` é um tipo privado?**

Para que nenhum outro pacote consiga ler ou sobrescrever o valor por engano.

### Ordem de leitura do código

1. `deploy/keycloak/wagering-realm.json`
2. `internal/infra/auth/principal.go` e `verifier.go`
3. `internal/infra/httpapi/auth.go` e o `NewRouter` em `server.go`
4. `internal/infra/auth/verifier_test.go`
5. `internal/infra/httpapi/auth_test.go`

SPEC 10

# Outbox e publicação de eventos

Desde a spec 06 cada mudança de saldo grava eventos na tabela `outbox_events`, na mesma transação. Agora um worker tira esses eventos do banco e os publica numa fila SQS, sem perder nenhum e sem trocar a ordem dos eventos de uma carteira.

## 1. O que a spec pedia

- Nenhum evento sai antes do commit; nenhum evento confirmado se perde.
- Vários publishers em paralelo sem publicar o mesmo evento ao mesmo tempo.
- Retry com backoff; trabalho de instância morta retomado por outra.
- Republicação mantém o mesmo `eventId`.
- Fila `wager-events.fifo`, com `MessageGroupId` = carteira e `MessageDeduplicationId` = `eventId`.

> **Eliminatório.** Publicar um evento antes do commit.

## 2. Por que a outbox existe

A tentação é publicar na fila logo depois de gravar no banco. Os dois são sistemas separados, e não há transação que cubra os dois:

| ORDEM | FALHA NO MEIO | RESULTADO |
|---|---|---|
| Publica, depois commita | O commit falha | Evento de um saldo que nunca mudou |
| Commita, depois publica | O processo morre antes de publicar | Saldo mudou e ninguém fica sabendo |

A **transactional outbox** resolve gravando o evento numa tabela do mesmo banco, na mesma transação do saldo. Ou os dois existem, ou nenhum. Publicar vira uma segunda etapa que pode ser repetida até dar certo.

1. O caso de uso grava saldo, lançamento e evento numa transação só.
2. O worker lê os eventos não publicados, envia ao SQS e marca como publicados.

## 3. At-least-once

Se o processo morrer depois de enviar e antes de marcar, o evento será enviado de novo. Não existe forma de evitar isso por completo: entre "enviei" e "anotei que enviei" sempre há um instante de risco. Por isso a garantia é **pelo menos uma vez** (*at-least-once*), e o consumidor precisa ser idempotente, deduplicando pelo `eventId`.

O SQS FIFO ajuda: mensagens com o mesmo `MessageDeduplicationId` em menos de 5 minutos são descartadas. Depois disso, a duplicata passa.

## 4. O problema da ordem

Numa fila FIFO, mensagens do mesmo `MessageGroupId` são entregues na ordem em que foram **enviadas**. Com o grupo sendo a carteira, quem consome vê os saldos de uma carteira na sequência certa. Mas só se nós enviarmos na sequência certa.

Com dois publishers reservando eventos soltos, o publisher A poderia pegar o evento 1 da carteira e o B o evento 2; se B for mais rápido, o 2 sai antes. Por isso a reserva é feita por carteira inteira.

## 5. A reserva por carteira

```
WITH heads AS (
    SELECT o.aggregate_id FROM outbox_events o
     WHERE o.published_at IS NULL
       AND o.next_attempt_at <= $1
       AND (o.locked_until IS NULL OR o.locked_until <= $1)
       AND NOT EXISTS (                    -- é o mais antigo não publicado da carteira
            SELECT 1 FROM outbox_events p
             WHERE p.aggregate_id = o.aggregate_id
               AND p.published_at IS NULL AND p.sequence < o.sequence)
     ORDER BY o.sequence
     LIMIT $3
       FOR UPDATE SKIP LOCKED
), claimed AS (
    SELECT id FROM (
        SELECT e.id, row_number() OVER (PARTITION BY e.aggregate_id ORDER BY e.sequence) AS
position
          FROM outbox_events e JOIN heads h ON h.aggregate_id = e.aggregate_id
         WHERE e.published_at IS NULL) ranked
     WHERE position <= $4
)
UPDATE outbox_events SET locked_until = $2      -- agora + 30s
 WHERE id IN (SELECT id FROM claimed)
RETURNING ...
```

- `heads` escolhe a **cabeça** de cada carteira: o evento não publicado mais antigo. Só ela é travada.
- Outro publisher rodando ao mesmo tempo pula a cabeça travada (`SKIP LOCKED`) e também não pega o evento 2, porque para ele a cabeça ainda está não publicada e o `NOT EXISTS` exclui o 2.
- `claimed` pega os eventos não publicados dessas carteiras, até `$4` por carteira. `row_number() OVER (PARTITION BY ...)` é uma **função de janela**: numera as linhas dentro de cada carteira, sem agrupar.
- O `UPDATE` grava a reserva de 30 segundos, a mesma ideia de *lease* da spec 08. Depois do comando, é ela que mantém os outros afastados.
- CTE (`WITH ... AS`) é uma consulta com nome, usada pelas partes seguintes do mesmo comando.

Uma cabeça com `next_attempt_at` no futuro (esperando backoff) segura a carteira inteira. É intencional: o evento seguinte não pode passar na frente.

## 6. Publicar e lidar com falhas

```
func (uc *PublishOutbox) publishInOrder(ctx context.Context, group []OutboxRecord) error {
    for i, record := range group {
        if err := uc.publisher.Publish(ctx, record.Message); err != nil {
            return uc.giveBack(ctx, record, group[i+1:], err)
        }
        if err := uc.outbox.MarkPublished(ctx, record.Message.ID, uc.clock.Now()); err != nil {
            return fmt.Errorf("mark event %s published: %w", record.Message.ID, err)
        }
    }
    return nil
}
```

| SITUAÇÃO | O QUE ACONTECE |
|---|---|
| Envio aceito | `published_at` preenchido, reserva liberada |
| Envio falhou | Conta a tentativa, guarda o erro, agenda com backoff; libera os eventos seguintes da carteira sem tentar; as outras carteiras seguem |
| Enviou e não conseguiu marcar | Depois da reserva, envia de novo com o mesmo `eventId` |
| Instância morreu depois de reservar | A reserva vence e outra instância assume |

- `group[i+1:]` é um *slice* do restante: os eventos que vinham depois do que falhou.
- Não há limite de tentativas. Descartar um evento confirmado é exatamente o que a outbox existe para impedir; o erro fica em `last_error` para quem for investigar.
- O registro da falha usa `context.WithoutCancel(ctx)`: se o desligamento cancelar o contexto no meio, a falha ainda é anotada e a carteira não fica presa por 30 segundos.

## 7. A mensagem no SQS

```
_, err := p.client.SendMessage(ctx, &sqs.SendMessageInput{
    QueueUrl:               aws.String(p.queueURL),
    MessageBody:            aws.String(string(message.Payload)),
    MessageGroupId:         aws.String(message.AggregateID),
    MessageDeduplicationId: aws.String(message.ID),
    MessageAttributes: map[string]types.MessageAttributeValue{
        "eventType": {DataType: aws.String("String"), StringValue:
aws.String(message.EventType)},
        "eventId":   {DataType: aws.String("String"), StringValue: aws.String(message.ID)},
    },
})
```

- SDK oficial `aws-sdk-go-v2`. Os campos são ponteiros (`*string`) para distinguir "vazio" de "não informado"; `aws.String` cria o ponteiro.
- `BaseEndpoint` aponta o cliente para o LocalStack; sem ele, o mesmo código fala com a AWS de verdade.

- As credenciais vêm da cadeia padrão da AWS (variáveis `AWS_ACCESS_KEY_ID`, perfis, papéis de instância), nunca do código.

## 8. Um worker genérico

O laço com `time.Ticker` da spec 08 foi extraído para `worker.Poller`, que recebe a tarefa como função:

```
type Job func(ctx context.Context) (more bool, err error)
```

O resolvedor de referências e o publicador da outbox viram duas instâncias do mesmo `Poller`, cada uma na sua goroutine. Em Go uma função é um valor como outro qualquer, e passar uma função costuma substituir a interface de um método só.

## 9. LocalStack

- Emula serviços da AWS localmente. No Compose sobe só o SQS, na porta `4566`.
- `deploy/localstack/init-queues.sh` roda quando o LocalStack fica pronto e cria `wager-events.fifo`.
- O healthcheck só fica verde depois que a fila existe; a aplicação espera por isso.
- O LocalStack não guarda as filas entre reinícios: parar o container apaga as mensagens.

## 10. Como foi testado

| CENÁRIO | RESULTADO VERIFICADO |
|---|---|
| Duas carteiras, 14 eventos | Todos enviados uma vez, na ordem de criação de cada carteira; payload enviado igual ao gravado |
| Segundo evento de uma carteira falha | O terceiro não sai antes dele; a outra carteira sai inteira; tentativa 1, erro gravado, nova tentativa em 1s |
| Publisher morre depois de reservar | Nada sai durante a reserva; depois dela, tudo sai |
| Enviou e perdeu a marcação | O evento sai de novo com o mesmo ID, depois o resto |
| 4 publishers, 5 carteiras, 90 eventos, lotes pequenos | 90 envios, nenhum duplicado, cada carteira em ordem |
| Publisher contra o LocalStack | Grupo e deduplicação corretos; reenvio do mesmo `eventId` descartado |

- Removi de propósito o `NOT EXISTS` e depois o `SKIP LOCKED`: os testes de ordem e de concorrência falharam nos dois casos, e voltei o código.
- Ponta a ponta no Compose: carteira com três apostas gerou oito mensagens na fila, na ordem e com `walletVersion` 1, 2, 3, 4. Com o LocalStack parado, uma aposta foi aceita, o evento ficou com `attempts = 1` e o erro gravado; ao voltar, foi publicado.
- `go test -race` passa, inclusive o teste contra o LocalStack; cobertura de `internal/` em 88,9%.

## 11. Para fixar

**Por que não publicar direto depois do commit?**

Se o processo morrer entre os dois, o evento se perde. Na outbox ele fica no banco até ser publicado.

**O que significa at-least-once e quem paga por ele?**

O evento pode chegar mais de uma vez. O consumidor paga, deduplicando pelo `eventId`.

**Por que travar só a cabeça de cada carteira?**

Para que outro publisher não pegue um evento posterior e o envie antes do anterior.

**A deduplicação do SQS FIFO dispensa a do consumidor?**

Não. Ela vale por 5 minutos; um reenvio depois disso passa.

**Por que o evento que falha não tem limite de tentativas?**

Porque ele descreve algo que já aconteceu no banco. Descartá-lo seria perder a informação que a outbox protege.

### Ordem de leitura do código

1. `internal/infra/postgres/outbox_repository.go`: `ClaimDue`
2. `internal/application/publish_outbox.go`
3. `internal/infra/sqs/publisher.go` e `deploy/localstack/init-queues.sh`
4. `internal/infra/worker/poller.go` e `module.go`
5. `internal/application/publish_outbox_test.go`

SPEC 11

# Consumidor SQS, inbox e DLQ

A segunda porta de entrada: provedores também podem mandar operações pela fila `wager-transactions.fifo`. O resultado financeiro tem que ser o mesmo do HTTP, mesmo com a mensagem chegando duas vezes, o processo morrendo no meio ou o banco fora do ar.

## 1. O que a spec pedia

- Usar o **mesmo** caso de uso do HTTP, com `data.idempotencyKey` como chave.
- Inbox: gravar `(consumerName, messageId)` e o hash do corpo na mesma transação do saldo, ledger e outbox.
- Remover a mensagem da fila só depois do commit.
- Falha transitória: tentar de novo com backoff. Erro permanente ou tentativas esgotadas: DLQ.
- No `SIGTERM`, parar de buscar e terminar ou devolver o que está em andamento.
- Política de acesso nas filas, sem deixar de validar no consumidor.

## 2. Como o SQS entrega

Receber uma mensagem do SQS **não** a remove. Ela fica invisível por um tempo (o *visibility timeout*) e, se ninguém chamar `DeleteMessage` até lá, volta para a fila. Isso protege contra consumidor morto, mas cria a reentrega: a mesma mensagem pode ser processada duas vezes.

| CONCEITO | O QUE É |
|---|---|
| Visibility timeout | Quanto tempo a mensagem recebida fica escondida dos outros consumidores |
| Long polling | O receive espera até 20s por mensagens, em vez de voltar vazio na hora |
| `ApproximateReceiveCount` | Quantas vezes a mensagem já foi recebida |
| Redrive policy | Depois de `maxReceiveCount` recebimentos, o SQS move a mensagem para a DLQ |
| DLQ (*dead-letter queue*) | Fila onde ficam as mensagens que não deram certo, para alguém investigar |
| FIFO e `MessageGroupId` | Ordem garantida dentro de um grupo; grupos diferentes podem ser processados em paralelo |

## 3. A inbox

A inbox é uma tabela que lembra quais mensagens já foram tratadas. O ponto importante é que ela é gravada **na mesma transação** da mudança de saldo:

1. `INSERT ... ON CONFLICT DO NOTHING` em `inbox_messages`.
2. Linha já existia com o mesmo hash: a mensagem já foi tratada. Nada é refeito.

3. Linha já existia com outro hash: o produtor reutilizou o `messageId` para outra coisa. Erro permanente.
4. Primeira vez: roda o caso de uso (trava a carteira, idempotência, regras, ledger, outbox) e marca `completed_at`.
5. Commit. Só então `DeleteMessage`.

Se o processo morrer depois do commit e antes do delete, a mensagem volta, cai no passo 2 e é removida sem segundo débito. Se morrer antes do commit, o rollback apaga também a linha da inbox, e a reentrega começa do zero.

> **Por que `ON CONFLICT DO NOTHING`?** Um `INSERT` comum que bate na chave primária gera erro, e no PostgreSQL um erro aborta a transação inteira. Com `DO NOTHING` o comando só não insere, e a transação continua usável para consultar a linha existente.

### A inbox não é redundante com a idempotência?

Em parte, é. A chave de idempotência já impediria o segundo débito. A inbox acrescenta duas coisas: reconhece a mensagem repetida *antes* de rodar o caso de uso, e detecta o mesmo `messageId` com outro corpo, coisa que a chave de idempotência não vê.

### E a deduplicação do FIFO?

Ela descarta envios com o mesmo `MessageDeduplicationId` por 5 minutos. Não cobre reentrega (é a mesma mensagem voltando, não um envio novo), nem um reenvio depois de 5 minutos. A inbox não tem prazo.

## 4. Um caso de uso, duas portas

O `ProcessWagerTransaction.Execute` da spec 07 tinha três partes: validar e calcular o hash, abrir a transação (repetindo se perder uma disputa de chave única) e processar. Separei essas partes para o consumidor montar a sua versão:

```
err = withinTxRetryingRaces(ctx, h.tx, func(ctx context.Context) error {
    stored, first, err := h.inbox.Receive(ctx, WagerConsumerName, msg.MessageID,
msg.PayloadHash, now)
    ...
    result.ProcessWagerResult, err = h.process.process(ctx, msg.Wager, hash)   // mesmo código
do HTTP
    ...
    return h.inbox.Complete(ctx, WagerConsumerName, msg.MessageID, now)
})
```

- `process` é minúsculo, portanto privado do pacote `application`. O caso de uso da inbox está no mesmo pacote e pode chamá-lo; o pacote `sqs` não pode.
- A repetição recomeça uma **transação nova**, com a inbox incluída: depois de uma violação de unicidade a transação antiga está abortada.
- HTTP e SQS calculam o mesmo hash de negócio (spec 07). Se a mesma operação chega pelas duas portas, quem chega depois pega o lock da carteira, encontra a operação gravada e recebe um replay.

## 5. Decidindo o destino de cada mensagem

| RESULTADO | AÇÃO |
|---|---|
| Processada, rejeitada por regra de negócio, `PENDING_REFERENCE` ou já tratada | Remove |
| JSON inválido, provedor fora da lista, validação, carteira inexistente, conflito de chave ou de `messageId` | Manda para a DLQ com o motivo e remove |
| Qualquer outro erro (banco fora, deadlock, erro desconhecido) | Esconde de novo por 2s, 4s, 8s... até 5min |
| Interrompida pelo shutdown | Visibilidade 0: volta na hora para outra instância |

A regra para erro desconhecido é tentar de novo: na dúvida, outra tentativa é mais segura do que descartar. O `maxReceiveCount = 5` garante que isso termina: na sexta recebida, o SQS move a mensagem para a DLQ.

```
func permanent(err error) bool {
    var validation *domain.ValidationError
    if errors.As(err, &validation) {
        return true
    }
    for _, target := range []error{errInvalidMessage, application.ErrWalletNotFound, ...} {
        if errors.Is(err, target) {
            return true
        }
    }
    return false
}
```

`errors.Is` percorre a cadeia de erros embrulhados com `%w`; `errors.As` faz o mesmo procurando um **tipo**. Assim o consumidor classifica erros criados lá no repositório sem conhecer os detalhes.

## 6. Ordem por carteira

Um receive traz até 10 mensagens, possivelmente de várias carteiras. O consumidor agrupa por `MessageGroupId`, roda as carteiras em paralelo (`sync.WaitGroup` com `wg.Go`) e as mensagens de cada carteira em sequência.

Se a primeira mensagem de uma carteira falha, as seguintes não podem passar na frente. A AWS documenta que não entrega outra mensagem de um grupo enquanto uma dele está invisível. No teste, o LocalStack entregou. Por isso as seguintes são escondidas pelo mesmo tempo da que falhou e voltam junto com ela. Foi um teste que falhou que mostrou isso.

## 7. Desligando sem perder mensagem

O consumidor usa dois contextos:

| CONTEXTO | CANCELADO QUANDO | EFEITO |
|---|---|---|
| Busca | Logo no `SIGTERM` | O long polling é interrompido; nada novo é recebido |
| Processamento | 3s antes do prazo de parada do Fx | A transação faz rollback e a mensagem volta com visibilidade 0 |

Depois do tratamento, as chamadas ao SQS (remover, devolver, mandar para a DLQ) usam `context.WithoutCancel` com um limite de 5s. Sem isso, um cancelamento chegando logo depois do commit impediria o delete. Isso não causaria erro financeiro, porque a inbox reconhece a reentrega, mas a mensagem voltaria à toa.

No Compose, `docker compose stop app` encerrou em menos de 1 segundo, com código de saída 0.

## 8. Política de acesso às filas

O desafio pede que a mensageria seja protegida pelo broker. O script do LocalStack grava em cada fila uma política no formato do IAM:

| FILA | PERMISSÕES |
|---|---|
| `wager-transactions.fifo` | Provedores enviam; `wagering-service` recebe, remove e muda a visibilidade |
| `wager-transactions-dlq.fifo` | `wagering-service` envia; um papel de operador lê e redireciona |
| `wager-events.fifo` | `wagering-service` envia; `wallet-service` consome |

> **Limite honesto.** O LocalStack, na configuração usada aqui, grava as políticas mas não as aplica. Na AWS elas valem. Por isso o consumidor também confere o provedor numa lista (`WAGER_QUEUE_PROVIDERS`) e o caso de uso valida tudo como no HTTP.

## 9. Valores escolhidos

| PARÂMETRO | VALOR | MOTIVO |
|---|---|---|
| `maxReceiveCount` | 5 | Com o backoff, cerca de 1 minuto de banco fora antes da DLQ |
| Visibility timeout | 30s | Folga sobre uma transação; consumidor morto devolve a mensagem em até 30s |
| Lote | 10 | Máximo do SQS |
| Long polling | 20s | Máximo do SQS |
| Retenção da DLQ | 14 dias | Máximo do SQS, para dar tempo de investigar |

## 10. Como foi testado

| CENÁRIO | RESULTADO VERIFICADO |
|---|---|
| Mesma mensagem duas vezes | Um débito; a segunda vem como já tratada |
| Mesmo `messageId`, outro corpo | Conflito; saldo intacto |
| HTTP e SQS juntos, 20 rodadas com 4 disputas cada | Um débito por operação |
| Commit feito, delete não (consumidor "morre") | Reentrega reconhecida pela inbox; um débito |
| Banco fora (pool apontando para porta fechada) | Mensagem fica na fila; volta e é processada quando o banco volta |
| JSON inválido, provedor desconhecido, conflitos | Na DLQ com o motivo; nada na origem |
| Erro transitório sempre | Depois de `maxReceiveCount`, o redrive move para a DLQ |
| Primeira mensagem de uma carteira falha | A segunda espera; a outra carteira segue |
| Stop com mensagem travada | Ela volta na hora, no segundo recebimento |

- Testes de propósito com defeito: ignorando a inbox, quatro testes falharam; tratando todo erro como permanente, seis falharam. Voltei o código.
- Ponta a ponta no Compose: mensagem duplicada, HTTP e SQS juntos, três casos de DLQ e o PostgreSQL parado e religado; o saldo terminou certo em todos.
- `go test -race` passa no container; cobertura de `internal/` em 89,5%.

## 11. Para fixar

**Por que remover a mensagem só depois do commit?**

Se removesse antes e o commit falhasse, a operação se perderia. Depois do commit, o pior caso é uma reentrega, que a inbox absorve.

**O que a inbox faz que a deduplicação do FIFO não faz?**

Não tem prazo, cobre reentregas e detecta o mesmo `messageId` com outro corpo.

**Por que um erro desconhecido é tratado como transitório?**

Descartar é irreversível; tentar de novo não. O `maxReceiveCount` limita as tentativas.

**Por que `PENDING_REFERENCE` remove a mensagem?**

A pendência já está gravada no banco e o worker da spec 08 continua dali. A mensagem não tem mais nada a fazer.

**Por que dois contextos no consumidor?**

Parar de buscar e interromper o trabalho em andamento são decisões em momentos diferentes do shutdown.

### Ordem de leitura do código

1. `internal/application/handle_wager_message.go`
2. `internal/infra/postgres/inbox_repository.go`
3. `internal/infra/sqs/message.go`
4. `internal/infra/sqs/consumer.go`
5. `deploy/localstack/init-queues.sh`
6. `internal/infra/sqs/consumer_integration_test.go`

SPEC 12

# Reconciliação

O saldo da carteira é guardado numa coluna, e cada movimento também fica no ledger. Se tudo está certo, somar o ledger dá o saldo. A reconciliação faz essa conta sob demanda e avisa quando não bate, sem consertar nada.

## 1. O que a spec pedia

- `POST /wallets/:walletId/reconciliation`, só para o serviço interno.
- Saldo calculado = créditos − débitos do ledger, incluindo a abertura.
- `difference` = armazenado − calculado.
- Saldo e ledger lidos numa visão consistente.
- Divergência na resposta, no log e numa métrica. Nunca alterar o saldo.

## 2. O problema: duas leituras, dois instantes

A reconciliação faz duas consultas: lê o saldo da carteira e soma o ledger. Enquanto isso, apostas continuam chegando. No nível de isolamento padrão do PostgreSQL, **READ COMMITTED**, cada comando enxerga o que estava confirmado quando *ele* começou:

1. Reconciliação lê o saldo: 9904.00.
2. Uma aposta de 1.00 confirma: saldo 9903.00 e um lançamento novo no ledger.
3. Reconciliação soma o ledger: já inclui a aposta, dá 9903.00.
4. Resultado: diferença de 1.00. Uma divergência que não existe.

Isso não é teoria: rodei o teste de concorrência com `READ COMMITTED` e ele acusou várias divergências falsas de exatamente 1.00, o valor de cada aposta.

## 3. A solução: REPEATABLE READ

Em **REPEATABLE READ**, a transação tira uma "foto" do banco na primeira consulta e todas as leituras seguintes enxergam essa mesma foto, mesmo que outros confirmem mudanças no meio.

| NÍVEL | CADA LEITURA VÊ | USO AQUI |
|---|---|---|
| `READ COMMITTED` | O que estava confirmado quando o comando começou | Padrão de todas as operações; os locks de carteira cuidam da concorrência |
| `REPEATABLE READ` | O que estava confirmado na primeira consulta da transação | Reconciliação |
| `SERIALIZABLE` | Como se as transações rodassem uma de cada vez | Não usado |

```
func (m *TxManager) WithinSnapshot(ctx context.Context, fn func(ctx context.Context) error)
error {
    if inTx(ctx) {
        return errors.New("snapshot requested inside another transaction")
    }
    return m.within(ctx, pgx.TxOptions{IsoLevel: pgx.RepeatableRead, AccessMode: pgx.ReadOnly},
fn)
}
```

- `READ ONLY`: qualquer escrita dentro dela é recusada pelo próprio banco. "Não altera o saldo" deixa de depender só de o código estar certo.
- Dentro de outra transação seria erro silencioso: o isolamento já foi decidido pela transação de fora. Por isso recusa.
- Ler em `REPEATABLE READ` não trava ninguém. O PostgreSQL guarda versões das linhas (MVCC), então as apostas seguem enquanto a reconciliação lê a foto antiga.

> **Alternativa válida:** uma consulta só, juntando carteira e soma do ledger, também enxergaria um único instante, porque um comando sempre vê uma foto só. Escolhi a transação para manter cada repositório com sua consulta, e porque a spec pedia o isolamento explícito.

## 4. A conta

```
SELECT COALESCE(SUM(CASE direction WHEN 'CREDIT' THEN amount_minor ELSE -amount_minor END),
0)::bigint,
       count(*)
  FROM wallet_ledger_entries
 WHERE wallet_id = $1
```

- A abertura é um crédito no ledger (spec 06); por isso o exemplo do desafio tem `checkedEntries: 2`: abertura e aposta.
- `SUM` de `bigint` devolve `numeric`, que não estoura. O `::bigint` de volta falha se o total sair do limite, em vez de dar um número errado.
- `COALESCE(..., 0)`: sem lançamentos, `SUM` devolve `NULL`.
- A diferença usa `Money.Sub`, que já aceita resultado negativo e confere a moeda.

## 5. Log e métrica sem sujar a aplicação

O pacote `application` não conhece `slog` nem Prometheus. A reconciliação chama uma porta, e quem implementa decide como relatar:

```
type ReconciliationReporter interface {
    Report(ctx context.Context, reconciliation Reconciliation)
}
```

A implementação em `observability` incrementa o contador `wagering_reconciliations_total` com o rótulo `result` (`consistent` ou `divergent`) e, na divergência, escreve um log `ERROR` com carteira, saldos

e diferença.

| CONCEITO PROMETHEUS | O QUE É |
|---|---|
| Contador (*counter*) | Só sobe; o que interessa é quanto subiu num intervalo |
| Rótulo (*label*) | Divide um contador em séries: `{result="divergent"}` |
| Registro (*registry*) | Onde as métricas são cadastradas; `/metrics` publica o que está nele |

- As duas séries são criadas com 0 logo no início. Um alerta do tipo "aumentou nos últimos 5 minutos" precisa de um valor anterior para comparar.
- O registro é próprio, não o global do pacote: cada teste cria um limpo e confere o valor exato.
- `GET /metrics` nasceu aqui, público como os health checks. A spec 13 acrescenta as outras métricas.

## 6. Como foi testado

| CENÁRIO | RESULTADO VERIFICADO |
|---|---|
| Exemplo do desafio: 1000.00 e aposta de 25.00 | 975.00 dos dois lados, diferença 0.00, 2 lançamentos; contador `consistent` = 1 |
| Carteira aberta com zero | 0 lançamentos, consistente |
| Saldo adulterado direto na tabela (+5.00 e −5.00) | Diferença 5.00 / −5.00, log `ERROR` com a carteira, contador `divergent` = 1, saldo intacto depois |
| 4 escritores com 100 apostas e 2 reconciliadores em laço | Cerca de 650 reconciliações, nenhuma divergência |
| Mesmo teste com `READ COMMITTED` | Várias divergências falsas de 1.00 (prova de que o teste pega o defeito) |
| Escrita dentro do snapshot; snapshot dentro de transação | As duas recusadas |
| Token de provedor, sem token, carteira inexistente, id inválido | 403, 401, 404, 400 |

- Ponta a ponta no Compose com Keycloak real: exemplo do desafio, adulteração pelo `psql`, contador e log conferidos.
- `go test -race` passa no container; cobertura de `internal/` em 90,2%.

## 7. Para fixar

**Por que `READ COMMITTED` dá divergência falsa?**

Cada comando vê um instante diferente; uma aposta confirmada entre a leitura do saldo e a soma do ledger aparece só na segunda.

### O `REPEATABLE READ` bloqueia as apostas?

Não. O PostgreSQL mantém versões das linhas; a reconciliação lê a versão da sua foto e as escritas seguem.

### Por que a transação é `READ ONLY`?

Para o banco garantir que a reconciliação nunca escreve, não só o código.

### Por que a aplicação não incrementa a métrica direto?

Para o núcleo não depender de Prometheus nem de `slog`; ele chama uma porta e a infraestrutura decide.

## Ordem de leitura do código

1. `internal/application/reconcile_wallet.go`
2. `internal/infra/postgres/tx.go`: `WithinSnapshot`
3. `internal/infra/postgres/ledger_repository.go`: `Summarize`
4. `internal/infra/observability/metrics.go`
5. `internal/infra/httpapi/reconciliation_test.go`